Implementation Guide

Architecture notes, project conventions, implementation seams, and decision records.

Working draft

Architecture notes, project conventions, implementation seams, and decision records.

Packet Runtime Modernization

Status

Pre-reseed modernization chapter status. The early sections preserve the chapter history; the current state has seeded Definition/Bundle packet material, closed in-scope runtime genericization, packet-based policy/dependency semantics, retired legacy bridge write intents, and trusted signed Preference.element writes.

Summary

This chapter brings the packet system, trusted runtime coordination, runtime connectors, and mutation orchestration up to the Preference-as-template direction. Definition, Bundle, and Preference are now canonical packet types; Preference remains the working runtime example with a signed trusted write path and runtime connector coverage for comparison. The broader goal is to keep every live packet type visible through the same coverage map while moving mutations into standardized Trusted Runtime Coordinator paths.

The work should preserve current behavior while replacing hidden assumptions with typed registries, explicit missing coverage items, and tests that fail when the modernization map drifts.

Modernization Targets

- packet types should have body schemas, compatibility entries, builder support, manifest definitions, definition parts, and runtime connector status recorded in one audit surface
- runtime mutations should map to coordinator responsibilities, prepare/finalize behavior, policy action IDs, signed corridor use, and interface event enrollment status
- Definition, Bundle, and Preference are canonical packet types with body schemas, compatibility entries, builder support, definition parts, and seed/profile coverage

- imported Definition and Bundle packets describe semantics but never introduce executable server behavior
- Trusted Runtime Coordinators and the Interface Event Coordinator should become the standard orchestration path after coverage is complete

Phase Plan

1. Save the broad plan and add coverage audits. 2. Use the audit output to prioritize packet-type definition work. 3. Expand manifest definitions and definition parts type by type, preserving existing schemas and compatibility behavior unless a type-specific schema evolution is explicitly approved. 4. Adapt runtime mutation paths into Trusted Runtime Coordinator seams behind the Interface Event Coordinator, keeping signed corridor behavior intact until each mutation has a tested replacement boundary. 5. Enroll completed types and connectors only when their docs, tests, policies, and runtime behavior are all aligned. 6. Retire planned-gap records only when the implementation and tests prove the gap is actually closed.

First Pass

The first pass is intentionally non-behavioral:

- add this chapter document
- link it from the implementation-guide index
- add packet type modernization coverage audits
- add runtime mutation modernization coverage audits
- add tests proving current types, mutation intents, Preference connector enrollment, and Definition/Bundle next-phase targets are all visible
- keep known modernization gaps green only through explicit planned-gap records

Guardrails

- historical baseline: Definition and Bundle were initially kept out of PACKET_TYPES; the current promotion pass enrolls them as canonical packet types
- do not change MutationIntent, route payloads, packet schemas, or compatibility registry behavior during the audit pass
- do not migrate legacy scope-display caches until a dedicated compatibility pass
- the initial audit baseline kept Preference.element out of signed mutation enrollment; the current corridor now enrolls preference.element.set through prepare/finalize
- do not rebuild generated public docs artifacts as part of this first pass

Test Plan

- run packet modernization coverage tests
- run runtime mutation modernization coverage tests
- keep packet-definition manifest and audit tests green
- keep signed route and service-writer audit tests green
- validate docs with npm run docs:validate

Decision Notes

The audit modules are the working checklist for the next implementation passes. Broken current invariants should fail tests. Expected gaps should remain visible as planned modernization work until they are closed by a later pass.

Trusted Runtime Coordinator Architecture

Current modernization direction is to organize secure runtime work around enrolled Trusted Runtime Coordinators rather than route-local write code.

Target coordinator families:

- Interface Event Coordinator for client-side UI event shaping before requests leave the interface
- Trusted Dispatch Coordinator for user/API intent routing, request normalization, and fail-closed preflight
- Trusted Definition Coordinator for active definition context, node definition preferences, definition part selection, compatibility-only definitions, and reseed readiness views
- Trusted Regulation Coordinator for policy and governance resolution
- Trusted Planning Coordinator for defaults, dependency, and bundle-plan resolution
- Trusted Building Coordinator for producing packet candidates through the generic builder pipeline
- Trusted Inspection Coordinator for structural refs, dependency satisfaction, and bundle coherence
- Trusted Testing Coordinator for schema, compatibility, and policy assertions
- Trusted Certification Coordinator for signatures, tickets, digests, and final integrity
- Trusted Archive Coordinator for packet-store writes, archive reads, refs, edges, and query indexes
- Trusted Verification Coordinator for local packet verification and verification reports

- Trusted Compatibility Coordinator for runtime schema-version posture, adapter-path metadata, and compatibility coverage audits
- Trusted Exchange Coordinator for bundle ingress/egress, import previews, export wrappers, shallow merge plans, and rebundle previews
- Trusted Projection Coordinator for UI-ready graph projections and available actions

These coordinators are runtime concerns. They execute trusted local code and feed live context through the Core Contracts Vault. Packet definitions may describe allowed operations, defaults, dependencies, policy requirements, actions, and projection hints, but imported packet definitions must never execute local runtime behavior. The Interface Event Coordinator is the client-side event former; it has no trusted authority. The Trusted Dispatch Coordinator is the runtime front desk: it normalizes requests, preflights registered client/API intents, and hands accepted runtime requests to downstream coordinators. The older Interface Signal Conductor and Trusted Request Coordinator names remain compatibility bridges only. The Trusted Definition Coordinator is the gate for definition lookup: internal candidate listing, ranking, conflict audit, compatibility selection, and runtime-view compilation functions are routed through its public coordinator surface instead of being imported as loose helper functions. The Trusted Regulation Coordinator follows the same gated coordinator pattern for policy contexts, write-policy gates, requirement listing, and readiness audits. The Trusted Planning Coordinator owns packet operation plans, builder selection, defaults, dependencies, child-plan seams, body-input plans, and planning readiness so default/dependency work does not masquerade as policy enforcement. The Trusted Building Coordinator is now the gated candidate materialization seam: it consumes trusted operation plans, builds packet candidate graphs, preserves Definition-part body candidate construction, and does not re-resolve policy/default/dependency DSL. The Trusted Inspection Coordinator is now the quality gate after Building: it inspects build results against frozen operation plan snapshots, validates candidate bodies against packet body schemas, checks plan/candidate graph alignment, and does not re-plan, sign, certify, or archive in normal mode. The Trusted Certification Coordinator now opens the post-inspection handoff: it hashes the plan, build result, inspection report, and candidate graph, creates a short-lived certification ticket, prepares dispatchable signature requests, verifies signed returns, and emits an archive-ready certified packet set without writing to storage. The Trusted Archive Coordinator is now the storage seam: it owns packet-store writes, archived packet reads, revision resolution, edge queries, archive search rows, and low-level bundle export primitives so Exchange, Projection, Verification, and Compatibility work can ask Archive instead of reaching into SQLite directly. The Trusted Verification Coordinator is now the verification seam: it owns packet, batch, bundle, archive-set, lineage, ref-closure, and certification-handoff verification while reading stored packet material through Archive instead of SQLite. The Trusted Compatibility Coordinator is now the schema-version posture seam: it calls core compatibility helpers for read adaptation, write preparation, adapter-path metadata, and registry summaries, then cross-checks Definition packet_compatibility descriptors without executing imported adapter code. The Trusted Exchange Coordinator is now the packet-movement seam: it normalizes incoming bundle shapes, asks Compatibility whether packet material is readable, asks Verification for packet/bundle posture, asks Archive for local revision comparison and low-level export, and returns non-mutating import/export/merge/rebundle plans. Pass A intentionally does not commit imported bundles or own storage writes. The Trusted Projection Coordinator is now the read-model seam: it asks Archive for packet material and edges, asks Definition for projection descriptors, and returns UI-safe packet, list, and graph view models without owning storage or hardcoding future surface layouts.

Interface Event Coordinator and Dispatch Intake

The first interface-to-runtime orchestration pass adds a Nexus app-layer InterfaceEventCoordinatorProvider and useInterfaceEventCoordinator() hook. UI handlers can now describe an event source, client intent, optional mutation intent, visual loading scope, local validation, dispatch callback, and refresh callback as one lifecycle. The coordinator creates an interface.event envelope, runs caller-owned validation, activates scoped loading, dispatches the work, normalizes success/failure results, and records recent event state for debugging.

Client validation is intentionally advisory. The Interface Event Coordinator supports required values, length checks, regex checks, and caller predicates, but runtime policy, proof, tickets, signing, persistence, and mutation effects remain authoritative downstream.

Runtime intake now exposes trustedDispatchCoordinator as the canonical front desk. It currently delegates basic request normalization to the existing foldered Trusted Request Coordinator implementation, preserving request compatibility while establishing dispatch naming. Mutation prepare/finalize routes enter Dispatch-owned write lifecycle methods rather than the legacy mutation service route authority. Optional interface event metadata travels in headers so signed mutation payload schemas stay unchanged.

Definition-Driven Build And Projection Direction

Dispatch-Owned Write Pipeline

The live /api/nexus/mutations/prepare and /api/nexus/mutations/finalize routes now call trustedDispatchCoordinator.prepareEnrolledMutationWrite and trustedDispatchCoordinator.finalizeEnrolledMutationWrite. There is no separate Write Coordinator. Dispatch owns route-facing lifecycle orchestration, then asks the existing trusted coordinator organs to do their jobs.

The intended chain is:

- Dispatch remains request and client-intent intake.
- Definition resolves active packet/workflow definitions.
- Regulation resolves policy and write-gate requirements.
- Planning resolves the operation plan.
- Building materializes candidates.
- Inspection validates candidates against the frozen plan.
- Certification owns signing tickets and certification readiness.

- Verification owns signed packet legitimacy.
- Archive stores certified packet sets.

The current correction intentionally blocks rather than falling back where the coordinator chain is incomplete. relation.follow.add and relation.association.add now close the first full route-facing Relation chains: Building emits archive-ready packet envelopes, Certification records expected packet digests on its tickets and certifies returned signed packet bundles, Verification validates the signed packets, and Archive stores the certified packet sets. Other enrolled write intents still need the same capability closure before they can leave blocked readiness.

The same declaration language should guide both packet creation and packet projection.

Creation uses definitions, builders, defaults definitions, dependencies definitions, policy requirements, and runtime variables to produce trusted operation plans before packet candidates are built. Projection should eventually use definitions, projection hints, component keys, action keys, display fields, and graph relationships to produce safe UI-ready view models.

Resolver ownership is split by domain:

- definition resolves active definition context, definition parts, node runtime preferences, compatibility-only definitions, and runtime definition views
- planning resolves operation plans, builder selection, default packet cascades, dependency satisfaction, workflow dry-runs, and child packet/component seams
- regulation resolves policy requirements, write-policy gates, governance rules, trust gates, moderation rules, voting eligibility, and other policy checks that may be needed inside or outside packet creation
- building creates packet candidates through the generic builder pipeline
- inspection validates candidate graphs and body candidates against the frozen operation plan snapshot
- projection resolves surfaces, display models, archive-backed packet lists, graph read models, badges, and available actions

Builders remain packet anatomy. Defaults describe normal starting shape. Dependencies describe required structural refs. Planning assembles those pieces into a concrete operation plan and asks Regulation for the active policy envelope when policy meaning matters. Regulation does not build packets or apply defaults; it classifies policy requirements and write gates for creation, projection, import/export review, moderation, runtime reads, and governance checks. Projection definitions describe safe display and interaction hints. The current Projection Coordinator keeps layouts intentionally basic, but its API is now shaped for definition-driven card fields, detail sections, edge groups, preferred surfaces, action slots, and later component/container descriptors.

Trusted Coordinator Scaffold Standard

Trusted coordinators now share a scaffold contract: public coordinator object, stable coordinator id, typed result envelope, issues, trace entries, optional request/operation ids, optional process-chain diagnostics, and a manifest entry describing expected methods. Foldered trusted coordinators expose only their public coordinator and public types from index.ts; internal function modules and registries stay private behind the coordinator surface.

npm run audit:trusted-coordinators checks the scaffold manifest. The audit currently treats Dispatch, Request, Definition, Regulation, Planning, Building, Inspection, Certification, Archive, Verification, Compatibility, Exchange, and Projection as foldered gated coordinators. There is intentionally no separate Write Coordinator; Dispatch owns write lifecycle orchestration. Resolution remains legacy-flat with a warning until it is promoted.

Trusted Process Chains and Issue Taxonomy

Trusted runtime process chains are lightweight runtime diagnostic objects, not automatic packet writes. A chain records the stage-by-stage path through a coordinator operation: coordinator id/kind, operation name, status, timestamps, summary artifacts, completed/failed/blocked/skipped work, child chain ids, and normalized issue codes. Stage snapshots are summary-only by default and must not carry raw packet bodies, signed request payloads, private material, or raw bundle contents.

Issue codes now have a canonical dotted taxonomy such as archive.packetnotfound, exchange.importcommitblocked, verification.packetstructuralinvalid, and certification.ticket_invalid. Existing underscore-style codes remain registered as legacy aliases so older coordinator code can migrate incrementally while reports and future interface handling see stable canonical codes.

Process chains preserve partial-work posture without imposing one rollback law on every operation. Each chain records a completion policy such as preservepartial, atomicrequired, dryrunonly, or coordinator_defined. Archive write flows now record successful writes, failed writes, skipped candidates, and whether partial work was preserved; Exchange import commit chains preview/plan work, blocked downstream Archive work, and Archive import child results.

Report packets remain optional. The runtime can create a compact trusted process report draft from a chain, but v1 does not write or sign those report packets automatically. Future server-wide and Interface Event Coordinator adoption should consume the same chain and taxonomy helpers rather than inventing surface-specific error handling.

Runtime Cleanup And Bounded Contexts

The trusted coordinator layer is structurally complete enough that runtime cleanup now shifts from inventing new coordinator seams to making the server runtime map intuitive. runtime/nexus/server/* is moving from one flat folder into bounded-context homes for identity, discussion, reaction, locality, scope, Packet Explorer, Dispatch mutation flow, readiness reports, and small shared helpers.

The first cleanup pass is intentionally move-first and split-later. Moved implementation files keep behavior intact and old top-level paths remain as compatibility bridges so API routes, tests, and service callers can migrate gradually. Large services such as discussion, auth, query data, reaction, and locality directory should be split by responsibility only after their bounded-context home is established.

Runtime cleanup should continue to tighten crossings through the trusted coordinator front doors. Archive remains the storage access seam, Compatibility remains the schema-version seam, Verification remains the legitimacy/signature seam, Exchange remains the packet movement seam, Projection remains the UI read-model seam, and Dispatch remains the request intake seam. Warnings are acceptable while older service callers migrate; newly cleaned seams should gain audit coverage so direct storage/signature/compatibility/projection bypasses do not reappear.

Trusted Planning Coordinator Pass

The Trusted Planning Coordinator is now the runtime middleman for operation planning. It exposes gated methods for resolving operation plans, default plans, dependency plans, builder descriptor selection, child packet plan seams, and planning readiness audits. Defaults and dependencies moved out of the Trusted Regulation Coordinator because they are construction-planning inputs, not policy enforcement by themselves.

Trusted Building Coordinator Pass

The Trusted Building Coordinator is now foldered and gated. Its public surface builds from trusted operation plans, materializes generic packet body candidate nodes, builds candidate graphs from plan trees, preserves typed Definition-part body candidate construction for reseed readiness, and audits whether planned operations can produce packet candidates. Building consumes the body-input plan prepared by Planning and does not parse policy/default/dependency DSL directly. Signing, inspection, certification, and archive writes remain outside Building.

Trusted Archive Coordinator Pass

The Trusted Archive Coordinator is now foldered and gated as the runtime storage seam. Its public surface can store certified packet sets, read archived packets in adapted/raw modes, query archived packet cards from search rows, resolve preferred or requested revisions, query packet edges, export low-level bundle payloads, and audit packet-store access.

Archive does not certify, inspect, build, plan, or project packets. It expects Certification to hand it an archive-ready certified packet set and expects later Import, Export, Projection, and Verification coordinators to use Archive for packet-store access instead of calling SQLite directly. In the current pre-reseed state, archive writes intentionally block when a certified candidate graph only contains body candidates and no full packet envelopes. That gives us the right seam now without pretending the full reseed packet-envelope materialization step exists yet.

Trusted Verification Coordinator Pass

The Trusted Verification Coordinator is now foldered and gated as the runtime verification seam. Its public surface verifies one packet, packet batches, transport bundles, archive-backed packet sets, parent revision lineage, packet ref closure, and Certification handoff packages. It preserves the existing raw-envelope verification law by checking structure and compatibility before signature assessment while passing the original raw packet material into the cryptographic verifier.

Verification does not store reports, write summaries, import bundles, export bundles, project UI models, or reach directly into SQLite. Stored packet verification flows must ask Archive for raw/adapted packet material, then run verification from that returned material. Local report writing and cached verification summaries remain service-layer responsibilities until report generation itself becomes a dedicated trusted coordinator seam.

Trusted Compatibility Coordinator Pass

The Trusted Compatibility Coordinator is now foldered and gated as the runtime schema-version posture seam. Its public surface can resolve one packet’s compatibility posture, adapt packet material for trusted reads, prepare versioned writes, resolve non-executable adapter-path metadata, compile registry/Definition compatibility profiles, audit packet-type coverage, and audit coordinator readiness.

Compatibility does not own adapter implementations, mutate packet material in storage, sign packets, verify cryptographic identity, import/export bundles, or promote imported Definition code into executable runtime behavior. Core schema compatibility remains the source of adapter logic. Definition packet_compatibility parts remain descriptive runtime metadata. This coordinator ties those two truths together so later Exchange, Projection, and reseed flows can ask one seam whether a packet can be interpreted, prepared, migrated, or blocked.

Trusted Exchange Coordinator Pass A

The Trusted Exchange Coordinator is now foldered and gated as the packet movement seam. Its public surface can preview imports, plan import commits, export packet sets through Archive, plan shallow merges, preview normalized rebundles, and audit Exchange readiness. It composes Archive, Verification, and Compatibility instead of reaching directly into SQLite, parsing schemas locally in routes, or letting import/export behavior drift into UI code.

Pass A originally stayed non-mutating for ingress. The caller migration pass added the minimum commit seam needed by Packet Explorer: Exchange now plans the import commit and delegates low-level bundle storage to Archive, while the Explorer service preserves import reports, preferred-head repair, validation modes, and response payload shape. The follow-up hardening pass keeps Archive as the only storage owner but narrows commits to Exchange-accepted revisions only, requires explicit acknowledgement for verification or compatibility risk, normalizes JSON string and archive-byte bundle inputs before Verification, and records planned/imported/skipped/missing revision keys for receipt comparison. Export delegates the low-level bundle bytes to Archive and wraps them in an Exchange manifest. Rebundle preview still normalizes packet material into a transport-shaped object and manifest without persisting anything.

Trusted Runtime Coordinator Audit And Caller Migration Pass

The trusted coordinator scaffold audit now checks foldered public surfaces, top-level barrel exports, expected manifest methods, and canonical result kinds. Resolution remains the only accepted legacy-flat warning. The audit also prints non-failing migration notes for runtime server callers that still use sensitive legacy seams, so caller cleanup can continue without confusing known transition points with scaffold failures.

Foldered Definition, Regulation, and Planning functions now report their manifest coordinator kinds instead of older transitional aliases such as workflow, policy, defaults, dependency, or builder. Those aliases remain in the shared kind union only for older legacy-flat workflow paths until a later compatibility cleanup.

Packet Explorer bundle export now routes through Trusted Exchange, which delegates bundle bytes to Trusted Archive. Packet Explorer import commit now routes through Trusted Exchange, which delegates low-level bundle import to Trusted Archive while the existing Explorer service preserves import reports, validation modes, preferred-head repair, and response payload shape. The legacy verification service now acts as a compatibility wrapper over Trusted Verification for packet assessment while it continues to own local validator identity and report-writing until those report flows move behind Certification and Archive.

Explorer's generic inspector payload now routes its revision resolution, raw/adapted packet read, graph edge lookup, scope labels, related packet labels, and read-model panel through Trusted Archive and Trusted Projection while preserving the existing API payload fields. readmodelview now carries the trusted packet projection view model instead of the legacy packet-interpreter output. Packet Explorer Search, Export, and Import still have their own service-specific logic and should be migrated only where payload parity is straightforward.

The current implementation is intentionally pre-reseed practical:

- default plans wrap definition defaults, policy-selected default refs, and local overrides without building packet bodies directly
- dependency plans gather Definition dependency parts, workflow dependency IDs, semantic descriptors, runtime capability refs, and optional workflow dry-run findings
- builder selection picks the best runtime-ready builder descriptor for the packet type, subtype, and action IDs
- child packet plans are a typed seam, currently empty until defaults, bundles, and projection/component layout semantics are declared enough to recurse safely
- operation plans call Regulation for policy contexts and optional write-policy gates, rather than duplicating policy logic

Regulation is now policy-only. It still works outside packet creation, including projection, import/export review, moderation, runtime reads, and governance checks. Planning may call Regulation during packet creation, but Regulation does not own defaults, dependencies, or builder selection.

Trusted Inspection Coordinator Pass

The Trusted Inspection Coordinator is now foldered and gated. Its public surface inspects build results, candidate graphs, individual packet body candidates, and plan alignment. Normal inspection receives the original Trusted Operation Plan snapshot plus the Trusted Build Result and asks whether Building faithfully materialized that plan. It validates candidate packet types, subtypes, builder IDs, planned body input values, body subtype, child candidate alignment, and packet body schemas. It does not resolve definitions, policies, defaults, dependencies, or operation plans again during normal inspection.

Inspection readiness intentionally runs the full Planning -> Building -> Inspection chain as an audit flow. That is different from judging an already-built candidate against a moving live context. Certification now owns the ticket/signature/hash handoff after Inspection; archival remains the later coordinator seam responsible for packet-store writes and indexes.

Trusted Certification Coordinator Pass

The Trusted Certification Coordinator is now foldered and gated. Its public surface prepares certification tickets, prepares signature requests, verifies signed ticket returns, certifies signed tickets into archive-ready candidate-set artifacts, and audits Planning -> Building -> Inspection -> Certification readiness. Certification receives the final build result and inspection report. It does not re-plan, rebuild, or re-inspect. Instead, it freezes the accepted artifacts into stable hashes so the interface can request a signature over the exact payload.

Certification owns the ticket/signature handshake, not storage. A successful signed return produces a certified packet set with the original candidate graph and hashes intact. Archive remains the later coordinator responsible for final packet-store writes and indexes.

Manifest Core Pass

The first chunky implementation pass expanded the packet manifest from the Preference template to the active packet types with generic builder-pipeline support: Element, Location, Role, Claim, Relation, Report, Proposal, Reaction, Decision, Action, Discussion, and Policy.

This pass remains runtime-ready. The new definitions describe existing body schemas, compatibility registry posture, generic builder support, action descriptors, planner descriptors, projection/index descriptors, and Definition parts. They do not change route payloads, packet schemas, runtime mutation behavior, or interface event connector enrollment.

Builder-missing types remain explicit missing coverage items in the modernization audit. Preference later received canonical builder support and signed-corridor write enrollment.

Packet-Type Authority Pass

The next implementation pass shifted the forward-looking checklist from legacy type enrollment toward manifest packettype authority. The later promotion pass enrolled Definition and Bundle as canonical packet types while preserving packettype language as the forward-facing semantic layer.

The pass added body builders for Definition, Bundle, and Preference. Those builders now feed canonical seed-profile helpers that create real Definition packet envelopes and a Bundle packet inventory for reseed material.

Packet-type modernization coverage is now the forward-looking audit surface for manifest definitions. The legacy type coverage remains as a migration bridge for live packet types and should keep missing coverage items visible until those types are converted into packet-type definitions and runtime connectors.

Compatibility Definition Standard Pass

The compatibility standardization pass makes compatibility a required, auditable definition contract. Every manifest packet type now needs a required packet_compatibility Definition part and a current-version identity adapter descriptor.

Generic type definitions derive definition compatibility descriptors from the canonical compatibility registry. Current-only types expose identity compatibility, while legacy-aware types expose adjacent upcast/downcast ladder edges where the registry has adapter functions. Multi-step ladders use the fullchainbundle strategy so future bundles can carry discoverable adapter metadata without pretending every adapter must touch the current schema directly.

The manifest audit now fails when compatibility posture and descriptors disagree, when downcast edges lack loss awareness, when duplicate adapter edges exist, or when a claimed full-chain graph is disconnected from the current version. This keeps reseed and import/export planning honest before runtime handler extraction or Trusted Runtime Coordinator pipeline promotion begins.

Mutation Handler Extraction Pass

Dispatch is now the route-facing authority for enrolled prepare/finalize orchestration. The Interface Event Coordinator remains the client/API-to-runtime event bridge; it does not own signed mutation internals.

The extraction pass introduces domain-composed mutation handler maps for locality, discussion, reaction, assembly, relation, role, and actor policy. MutationPrepareHandlers and MutationFinalizeHandlers remain compatibility facades for the current implementation, while the composed maps give the runtime a clearer stepping-stone toward generic packet planners.

Each live mutation intent now has a genericization classification:

- generic_ready means the intent is close to manifest/action/planner routing once its local read dependencies are isolated.
- plannerextractionneeded means reusable packet planning must be extracted before generic routing.
- workflow_specific means runtime orchestration still coordinates multiple packet operations or projections.
- legacy_bridge means the intent is a compatibility alias that should collapse into a canonical intent.

This pass originally preserved behavior while recording which mutation-service code should be retired, which should become reusable planners, and which orchestration remains runtime-owned before live trusted coordinator promotion. The old route executor files have since been removed; this section remains as the historical migration map.

Packet Operation Ontology Pass

The operation ontology pass adds the missing contract between packet definitions and trusted runtime execution. Packet definitions may now describe allowed mutation semantics by mapping their manifest mutation descriptors to known operation kinds such as singlepacket.create, singlepacket.revise, relation.set, claim.assert, reaction.set, bundle.import, projection.refresh, compatibility.adapt, and workflow.compose.

This ontology is an allowlist, not executable packet-defined code. Each operation records its expected planner kind, builder kind, result type, trusted local runtime engine, generic capability posture, and safety notes. Packet definitions can request known operation semantics, but only local trusted engines may execute builders, planners, adapters, workflows, or persistence.

The manifest audit now fails closed when a mutation descriptor cannot map to a known operation kind. The forward-looking packet operation modernization coverage lists every manifest mutation, the operation kinds it resolves to, the trusted local engine requested, and whether the gap is already mapped or still planned.

The signed mutation genericization audit also records operation mappings for every live mutation intent:

- generic_ready intents map directly to concrete operation kinds, but still wait for the later live promotion pass.
- plannerextractionneeded intents map to their target operation kind and keep an explicit planner extraction gap.
- workflow_specific intents map to workflow.compose or a composed operation set and remain runtime-owned until their component operations can be split safely.
- legacy_bridge intents point at the canonical operation direction they should collapse into.

This keeps ingress normalization separate from mutation authority. The Interface Event Coordinator can normalize client/API ingress requests and eventually choose operation descriptors, while the trusted mutation pipeline still owns prepare/finalize/proof/persistence until a later pass promotes selected operation kinds through the generic path.

Generic Workflow Planner Contract Pass

The workflow planner contract pass adds the declarative layer above individual operation kinds. Packet definitions may now describe definition workflow plans as ordered steps over known generic operations, trusted resolver IDs, value bindings, simple conditions, policy action IDs, and runtime dependency IDs.

Workflow plans are data, not code. Definitions can say "resolve actor and target, then run relation.set" or "if the input value is present run reaction.set, otherwise run reaction.clear." They cannot introduce arbitrary functions, dynamic imports, persistence behavior, route payloads, or proof rules. The runtime interpreter validates every operation, resolver, dependency, condition operator, policy action, and step reference against local allowlists before producing a dry-run plan.

The first definition workflow plans cover the generic-ready mutation candidates:

- relation.follow.add
- relation.follow.clear
- role_association.claim.set
- reaction.vote.set

These plans do not enroll live execution. They prove the manifest can describe packet-specific variables and ordered generic work while keeping Dispatch and the trusted write chain as the live prepare/finalize/proof/persistence authority.

Policy and dependency descriptors now matter as referenced workflow metadata, but their full semantics remain a dedicated pre-reseed pass. Unused legacy packet types remain explicit missing coverage items and do not block the generic workflow contract or switch-over planning.

Workflow Alignment Pass

The workflow alignment pass connects the manifest workflow contract to the extracted mutation planner map. It adds a runtime-side audit that lists every live mutation intent, its genericization status, operation mapping, workflow-plan coverage, policy action IDs, trusted resolver/capability IDs, and remaining packet-specific assumptions.

The alignment map is the working checklist for replacing packet-specific mutation-service history with coordinator-owned execution. Generic-ready intents must have clean workflow dry-runs and trusted local capability coverage. Planner-extraction intents may either have a definition workflow plan or an explicit missing coverage item. Workflow-specific intents remain runtime-owned orchestration until their component operations can be split safely. Legacy bridge intents point at canonical workflow directions rather than receiving independent workflows.

This pass expands definition workflow coverage for knowable planner-extraction candidates:

- relation.association.add

- relation.association.clear
- relation.residence.add
- discussion.reply.create

The alignment remains runtime-ready. Existing runtime planner modules are registered as trusted local capabilities by descriptor, but their implementation is not moved, rewritten, or invoked through generic execution yet. Unused removed packet types remain visible missing coverage items and do not block switch-over planning.

Runtime Ingress And Mutation Handoff Pass

Ingress preflight and mutation authority remain separate layers. Ingress preflight owns client/API normalization, manifest/workflow lookup, connector selection, definition handoff metadata, and response/refresh hints. Dispatch and the trusted write chain own policy/proof authority, prepare/finalize lifecycle, mutation tickets or their replacement, signed packet validation, persistence, and canonical mutation effects.

The handoff pass adds a definition PacketRuntimeMutationHandoff contract. A handoff records the normalized mutation direction, workflow alignment status, operation kinds, workflow plan IDs, trusted capability IDs, policy action IDs, dependency IDs, resolver IDs, prepare/finalize coordinator names, and return refresh hints. It carries externaldefinitionexecution_enabled: false to record that imported definitions describe behavior while trusted local runtime code executes it.

Generic-ready and workflow-aligned planner-extraction intents can now produce definition_ready handoffs. Runtime-owned workflow intents produce explicit non-ready handoffs with orchestration reason codes. Legacy bridge intents point at canonical handoff directions. Unknown mutation intents fail closed before any mutation handoff.

At the time of this pass, this did not change the live mutation routes. The current state is stricter: Dispatch is the route-facing signed-write authority, and authenticated Preference.element writes enter the Dispatch-owned write pipeline rather than the old mutation service route authority.

Packet-Based Policy, Dependency, and Client Ingress Enrollment Pass

Policy and dependency requirements are now audited as packet-backed semantics rather than a second runtime-only dependency system. Workflow plans can reference policy action IDs and dependency IDs, but those references must resolve to packet policy semantics, packet Definition dependency parts, operation ontology entries, trusted workflow resolvers, or trusted local capability metadata. Runtime descriptors may index and validate those references, but they do not define packet meaning.

Policy packets remain the semantic authority for live write-lock policy. MutationPolicyGate remains the live resolver for scope policy refs, actor security mode, proof level, and accepted proof methods. Definition workflow policy descriptors record how policy action IDs map back to that packet-based enforcement model, while manifest-only actions remain definition metadata until a later live write-policy enrollment pass.

Runtime ingress now has a client/API enrollment registry. The registry is an internal allowlist of adapter-originated transport routes and portable client intent IDs:

- /api/nexus/mutations/prepare enrolls the current signed mutation intents.
- /api/nexus/mutations/prepare enrolls the Preference.element client intent; authenticated shell preference writes now use the same prepare/finalize corridor as other claimed mutations, while /api/nexus/shell-preferences remains guest compatibility state only.
- each enrollment records route or transport source, client intent ID, mutation intent, operation kinds, workflow plans, policy actions, dependency refs, and current live mode.

Unknown or custom route/intent pairings fail ingress preflight. The preflight may resolve handoff metadata and packet-backed policy/dependency descriptors, but it does not authorize, ticket, sign, persist, finalize, or bypass Dispatch-owned trusted writes. This keeps the generic write path aligned with enrolled client/API ingress from web, device, automation, or other adapters instead of accepting arbitrary injected operation requests.

Interface-Neutral API Ingress Pass

The enrollment layer is interface-neutral. Web shell, Raspberry Pi controls, local automation, and future adapters should all map their local events into the same portable client intent IDs, such as scope.follow.set, scope.association.clear, discussion.reply.create, and preference.interface.set. Runtime registries should not encode web UI concepts as packet meaning.

The live API routes now consult ingress preflight before delegating to the live corridor:

- prepare parses the request intent, validates client/API ingress enrollment, then delegates to trustedDispatchCoordinator.prepareEnrolledMutationWrite;
- finalize delegates to trustedDispatchCoordinator.finalizeEnrolledMutationWrite, which certifies the returned signed packet bundle, verifies the signed packets, and stores the certified packet set through Archive; reaction-specific response decoration happens afterward in the reaction runtime adapter, not inside Dispatch;
- authenticated shell preferences use the standard prepare/finalize mutation routes with preference.element.set;
- /api/nexus/shell-preferences remains a guest compatibility route and is outside packet-runtime connector enrollment.

This pass is still not generic execution. Preflight validates allowlist and metadata alignment only; mutation policy/proof/ticketing/persistence remains authoritative.

No-Deferral Pre-Reseed Closure Program

Reseed design is now gated on full closure of in-scope live runtime modernization work. The chapter can still be split across multiple implementation passes, but the remaining live runtime work is no longer tracked as open-ended missing coverage items. The pre-reseed closure ledger classifies every live mutation intent, runtime connector path, workflow plan, policy/dependency requirement, client/API ingress enrollment, mutation handoff, and active packet type as closed, closingnow, queuedpre_reseed, or blocked.

The first proving promotion is Relation set/clear:

- relation.follow.add
- relation.follow.clear
- relation.association.add
- relation.association.clear

relation.follow.add, relation.association.add, and reaction.vote.set are now proven Dispatch-owned live writes. They prepare and finalize through the intended coordinator chain: Dispatch intake, Planning, Building, Inspection, Certification ticketing, signed-packet certification, Verification, and Archive storage. API routes, route payloads, response shapes, policy action IDs, packet schemas, proof behavior, signatures, and persistence semantics remain unchanged. reaction.vote.set response decoration is isolated in the reaction runtime adapter. relation.follow.clear and relation.association.clear remain queued for the same full-chain promotion rather than falling back to the legacy signed corridor.

The remaining pre-reseed queue is explicit:

- relation, claim, and reaction generic enrollment for association, home locality, role claim, packet signal, and role reaction paths
- discussion and locality workflow decomposition for reply/thread planners, default surfaces, locality path/graph planning, and assembly creation
- packet-based policy/dependency semantic authority so Policy packets and Definition dependency parts carry enough meaning for reseed
- legacy bridge retirement for compatibility aliases that should not survive into the fresh reseed world, now closed by removing legacy bridge intents from the live prepare corridor
- a final reseed readiness audit after all in-scope modernization closure items are closed

Unused never-live packet types were pruned from active canon. They can return only through the same definition, schema, builder, seed, and audit path as any other active type.

Generic Composite Workflow Adapter Pass

Complex graph workflows now use named trusted composite adapter shapes in definition. These adapters are local runtime-owned orchestration patterns that compose known packet operations, policy actions, dependency refs, and result metadata. Packet definitions and workflow alignment may reference adapter IDs, but packet definitions still cannot provide executable code.

The first adapter shape is composite.batch.packet_operations, used by locality.graph.apply. It describes the reusable graph pattern: resolve inputs, plan structural packets, plan relation operation batches, resolve grouped policy, prepare unsigned digests, carry prepared-result metadata, and classify projection/refresh side effects as runtime return extensions.

Two additional graph-style shapes are recorded for reuse:

- composite.defaultpacketset.ensure for idempotent default packet-set creation, first represented by discussion.surfaces.ensure.
- composite.entitycreate.withfollowups for a primary entity packet plus optional follow-up operations, first represented by assembly.element.create.

This pass remains runtime-ready. Live API routes, payloads, mutation ticketing, signing, persistence, projections, and response shapes remain unchanged. Complex workflows stay queued for live promotion until adapter parity tests prove prepare/finalize behavior against the current mutation oracle.

Remaining Runtime Genericization Closure Pass

The second live generic promotion expands the trusted workflow seam beyond follow relations. The direct operation paths now enrolled behind trusted planning/building work are:

- relation.follow.add
- relation.follow.clear
- relation.association.add
- relation.association.clear
- relation.residence.add
- role_association.claim.set
- reaction.vote.set

The live behavior contract is unchanged: API payloads, policy action IDs, ticketing, signatures, packet schemas, persistence, projections, and finalize handlers remain the current mutation authority. The prepare side now resolves these direct operations through trusted local generic planners for scoped Relation, role Claim, and packet-signal Reaction writes, using the current mutation planners/builders as the behavior oracle.

The remaining composed workflows now have named adapter shapes instead of open-ended gaps:

- composite.locality_path.create.v0 for reusable entity/path creation and directory projection refresh.
- composite.discussionthreadpost.create.v0 and composite.discussion_reply.create.v0 for canonical Discussion(subtype: post/message) writes.
- composite.role_reaction.set.v0 for mutual-exclusion support/dispute/clear reaction composition.
- composite.actorwritepolicy.update.v0 for actor-owned Policy packet revision plus actor projection refresh.

Discussion follow-up is closed for the fresh canon: new top-level discussion writes use Discussion(subtype: post) semantics, while replies use Discussion(subtype: message). DiscussionThread, DiscussionPost, and DiscussionReply are not active fresh packet types.

Initiative follow-up is also explicit: the fresh-reseed direction is Action(subtype: initiative) as the default OWA anchor for policy, template, branding, locality, voting, and governance defaults. Cause is not an active fresh packet type. This pass does not add an initiative selector or UI behavior.

Live Composite Workflow Promotion Pass

The runtime genericization lane is now closed for in-scope live prepare handling. Direct packet operations continue through the trusted generic operation seam, and composed workflows now prepare through trusted generic-composite workflow resolvers:

- composite.locality_path.create.v0
- composite.locality_graph.apply.v0
- composite.discussion_surfaces.ensure.v0
- composite.assembly_element.create.v0
- composite.discussionthreadpost.create.v0
- composite.discussion_reply.create.v0
- composite.role_reaction.set.v0
- composite.actorwritepolicy.update.v0

These resolvers execute trusted local runtime code only. Adapter descriptors describe the reusable workflow shape and audit metadata; packet definitions still cannot inject executable behavior. Dispatch is the route-facing signed-write authority; domain finalize handlers are removal targets, not a fallback path.

Actor write-policy update is mechanically promoted through the composite seam, and Policy packets plus Definition dependency parts are authoritative enough for the fresh genesis contract. Discussion canonicalization to top-level Discussion(subtype: post) plus reply Discussion(subtype: message) and OWA Action(subtype: initiative) are now part of the active reseed contract.

Initiative Action Hierarchy and Discussion Schema Readiness

The pre-reseed packet model now treats Action(subtype: initiative) as the forward initiative/work hierarchy anchor. Action packets can carry hierarchy refs plus packet-backed policy, template, and default packet-set refs so OWA defaults can be overridden without adding OWA-specific fields to Element or hardcoding defaults in runtime.

Canonical discussion shape now reserves Discussion(subtype: post) for top-level multimedia forum artifacts that start a thread, while Discussion(subtype: message) remains the reply/comment shape. Legacy thread/post/reply packet types are pruned from fresh canon.

Governance hooks remain schema-ready rather than workflow-complete: quorum, minimum trust, voter eligibility, approval thresholds, and voting gates should be expressed through packet-backed Policy/default material linked from the applicable initiative Action, scope, proposal, or definition context. Decision is the formal outcome packet; Report(subtype: decision_report) is reserved for future tally/evidence/process closure material.

Packet-Based Policy and Dependency Semantic Authority

Policy and dependency semantic authority is now closed for reseed readiness, while live governance execution remains later work. Policy current schema includes nullable defaultpolicy and governancepolicy sections. Older Policy revisions upcast those sections to explicit null; downcasts to older schema versions report loss when non-null default or governance material cannot be represented.

Policy packets are the semantic home for write locks, trust baselines, relation requirements, dependency and alignment rules, default inheritance, and governance hooks. The live write-lock path still runs through MutationPolicyGate; the new semantic helpers resolve and audit packet meaning without executing proposal/vote/decision behavior.

Definition dependencies_definition parts now carry meaningful dependency refs for packet operations, builder pipelines, action bridges, canonical packet-type builders, Preference projections, Bundle inventory building, and trusted compatibility/projection seams. Workflow and runtime dependency IDs must resolve through one of these anchors, Policy packet semantics, the operation ontology, a trusted workflow resolver, or an explicit trusted local engine contract.

The seeded OWA Action(subtype: initiative) now links to default-inheritance and governance-baseline policies. Forward default/policy resolution uses the Action initiative anchor.

Canonical Subtype Reset

The pre-reseed reset prunes inactive and legacy packet types from active canon. Fresh canon now includes only Definition, Bundle, Element, Location, Role, Claim, Relation, Report, Proposal, Reaction, Decision, Action, Discussion, Policy, and Preference.

Every active packet body uses top-level body.subtype as its packet classifier. Fresh writes reject old top-level classifier names such as kind, policykind, rolekind, proposalkind, claimkind, and reaction_value. Nested rule mechanics can still use precise names such as quorum or threshold kind when they are not packet classifiers.

Cause, Signal, separate initiative/work types, separate discussion thread/post/reply/forum/space types, Minutes, Artifact, and other pruned types are not valid fresh packet types. The alpha database is expected to be archived and wiped rather than adapted into fresh canon.

Final Pre-Reseed Wrap-Up

The final wrap-up retires the remaining live legacy bridge mutation intents from fresh writes:

- association.claim.set
- residence.claim.set

Canonical writes now enter through relation.association.add, relation.association.clear, and relation.residence.add. Historical legacy claim material remains readable/importable/projectable through compatibility surfaces, but the signed mutation prepare corridor, client ingress registry, handoff coverage, and live write-policy action list no longer enroll the legacy bridge intents.

The final readiness handoff lives in runtime audit code as createFinalPreReseedReadinessReport(). It records canonical write intents, compatibility-only legacy surfaces, OWA seed/default anchors, required default policies, discussion default packets, canonical definition packet types, and out-of-scope never-live packet types. Reseed design should start from that report rather than rediscovering chapter state from scattered modernization audits.

Definition Packetization and Preference Dispatch Promotion

Active manifest definitions now have canonical packet material. buildDefinitionPacketSeedEnvelopes() emits schema-validated Definition packet envelopes for every active manifest definition part, and buildDefinitionBundleSeedEnvelope() groups those envelopes into one Bundle.packet_set definition profile inventory. auditSeededPacketDefinitionProfile() compares that packet material back to the core manifest and fails on missing parts, duplicate or stale bundle refs, digest drift, or manifest/profile mismatch.

Definition, Bundle, and Preference are now first-class canonical packet types. The active definition profile is seeded as real packet material, but execution remains trusted-local: imported Definition or Bundle packets can describe schemas, operations, policies, dependencies, planners, and builders, but cannot introduce executable server behavior.

This is packetized seed truth, not imported-code execution. Stored Definition and Bundle packets may describe schemas, operations, policies, dependencies, planners, and builders; trusted local runtime registries remain the only executable authority.

Claimed Preference.element writes now use the signed mutation prepare/finalize path as preference.element.set. The client prepares through /api/nexus/mutations/prepare, signs the prepared Preference packet candidate, finalizes through /api/nexus/mutations/finalize, and then receives the same projected Preference result shape from the mutation result. /api/nexus/shell-preferences is now guest compatibility state only. The old direct preference.element.interface.set connector is retained as a definition/internal comparison bridge rather than the live claimed-write path.

Reaction packet convergence pass

Current pre-reseed canon collapses lightweight votes, packet signals, support/dispute posture, and emoji-style emotional responses into Reaction as the single packet type for target-agnostic responses.

Reaction packets are replaceable per actor/target/context. A revision can carry any combination of:

- vote_value: ’up’, ’down’, or null
- attestation_value: support, dispute, or null
- emoji_keys: a bounded list of basic emoji keys

Reaction does not encode target packet type or target-specific purpose. Proposal voting, discussion up/down signaling, role support/dispute posture, and later emoji/reaction UI should all route through the same packet type and projection layer. The old standalone Vote and Attestation packet types are removed from fresh canon for the clean pre-reseed path.

Trusted Resolution Coordinator and Projection Definition Pass

The trusted runtime coordinator family now has a shared home under runtime/trusted_coordinators/*. Direct generic workflow promotion, composite workflow promotion, composite adapters, resolution, and projection share this layer instead of living as one-off files under the Nexus server adapter folder.

A portable resolution DSL now lives in core as declaration language, not executable behavior. It defines shared binding shapes, resolution steps, and preset descriptors such as primitive bindings, packet refs, policy gates, dependency gates, relation lookup, discussion thread context, role scope context, compatibility projection, and UI card projection. Packet definitions and workflow descriptors may point at preset IDs, while trusted runtime coordinators decide how those presets are actually resolved locally.

Packet projection descriptors are now richer definition data. They can declare field bindings, layout/component keys, preferred surfaces, action registry keys, dependency IDs, policy action IDs, and resolver preset IDs. The generic packet definitions now seed default summary-card and detail-panel projections for each active packet type, giving the UI a definition-driven shape without embedding UI-specific packet law into core logic.

Definition part body building now carries descriptor payloads for action registries, builders, planners, workflow plans, projections, compatibility adapters, and dependency descriptors. This makes Definition packets more useful for reseed instead of merely listing IDs.

This pass is still conservative at the product edge. The shared packet action service now asks projection definitions for the preferred packet surface, but route-level UI layouts are not yet fully generated from projection descriptors. The next useful pass is to wire projection view models into universal packet card/detail components and reseed Definition packets with these richer descriptor bodies.

Runtime Responsibility Audit

Executive summary

This audit maps the current runtime toward its ideal ownership model. The trusted coordinator layer now has most of the right front doors, and Dispatch is the route-facing entrypoint for enrolled writes, but the API-facing runtime still contains product-specific service islands, legacy signed-corridor code, hardcoded packet type/subtype branches, and read-model logic that should gradually move behind Definition, Projection, Regulation, Planning, Certification, Archive, Verification, Compatibility, Exchange, and Dispatch seams.

Current scan baseline:

- runtime/*: 359 TypeScript files
- runtime/nexus/server/*: 138 TypeScript files
- runtime/trusted_coordinators/*: 165 TypeScript files
- Current trusted-coordinator audit notes report 10 direct storage-touch files, 3 direct signature-verification files, 1 API packet-parse crossing, and 24 legacy signed-corridor naming hits across 19 server/runtime files
- npm run audit:direct-storage-touches currently classifies 157 storage touches: 97 allowed reads, 44 allowed infrastructure touches, 16 migration-target touches that still need coordinator ownership, and 0 unclassified touches
- legacy signed-corridor naming is now confined to the scaffold audit guard and archived docs; runtime/UI code uses Dispatch wording

The near-term goal is not to hide every product concept from runtime. Runtime may understand generic packet anatomy, refs, schema posture, signatures, storage, projections, and operation results. The goal is to stop encoding product behavior as scattered runtime branches when the rule can be described by packet definitions, projection descriptors, coordinator capabilities, or OWA adapter/profile modules.

Guardrail housekeeping baseline

The trusted coordinator audit is now responsible for both scaffold shape and migration visibility. It checks for unmanifested trusted*coordinator folders, required package test scripts, manifest/barrel/method/result-kind drift, and registered trusted issue codes. Runtime crossing findings are reported as non-failing notes so migrations remain visible without blocking unrelated work.

The audit recursively scans implementation files under runtime/nexus/server/ and src/app/api/nexus/, not only top-level compatibility bridge files. Current note categories are direct storage touches, direct signature verification, direct packet interpretation, direct bundle import/export, packet parsing inside API routes, and legacy Dispatch corridor references. These notes are migration targets, not scaffold failures. The audit now also fails if removed legacy executor files such as mutation-service.ts, signed-packet-finalizer.ts, or the old Dispatch prepare/finalize handlers reappear.

Direct storage touches now have a separate classification guard in runtime/nexus/server/readiness/direct-storage-touch-audit.ts and a package script, npm run audit:direct-storage-touches. This audit scans runtime, storage, trusted coordinator, and Nexus API roots; it fails only on unclassified storage writes. Reads, Archive/storage internals, bootstrap/identity/verification infrastructure, and derived-state cache writes are classified separately from product-service packet writes that still need trusted coordinator migration. The final pre-reseed readiness report imports this audit so any newly unclassified storage mutation blocks the reseed handoff.

Trusted coordinator tests now have a package entrypoint: npm run test:trusted-coordinators. The combined guard command is npm run check:trusted-coordinators, which runs the scaffold/crossing audit before the coordinator test suite.

The empty trustedwritecoordinator remnant was removed. There is intentionally no Write Coordinator; write lifecycle orchestration belongs to Dispatch. Exchange documentation was also corrected to reflect its current import commit orchestration role while preserving Archive as the storage owner.

Ownership categories

Use these categories when classifying runtime modules and future migrations.

| Category | Meaning | Ideal location | | --- | --- | --- | | trustedcoordinatorfrontdoor | Public coordinator surface and manifest-visible method owner | runtime/trustedcoordinators/<coordinator>/ | | trustedcoordinatorinternal | Private coordinator registry, function, type, and internal helper | runtime/trustedcoordinators/<coordinator>/ | | definitiondrivencandidate | Hardcoded behavior that should become Definition, defaults, dependency, builder, action, or projection metadata | Usually Definition/Planning/Projection plus packet definitions | | projectioncandidate | Read-model logic that should move behind Trusted Projection while preserving API payloads | runtime/trustedcoordinators/trustedprojectioncoordinator/ or projection adapter | | storageadapter | Concrete persistence adapter, schema, bundle storage primitive, or query adapter | runtime/storage/ or Archive-facing storage adapter | | runtimeadapter | API-facing glue that composes coordinators and storage without owning packet semantics | runtime/nexus/server/ bounded context | | owaproductadapter | OWA-specific profile behavior, civic defaults, locality conventions, or compatibility policy | future runtime/nexus/server/adapters/owa/ or clearly named product module | | compatibilitybridge | Old import path, legacy read fallback, compatibility projection, or temporary wrapper | nearest bounded context with bridge docs | | legacysignedcorridor | Former Dispatch prepare/finalize/ticket/proof code that must be retired rather than wrapped | replacement through Dispatch, Certification, Verification, and Archive | | testoraudit | Tests, readiness ledgers, modernization reports, and static audits | near tested subsystem or runtime/nexus/server/readiness/ |

Current runtime map

| Area | Current responsibility | Ideal classification | Ideal direction | Risk | | --- | --- | --- | --- | --- | | runtime/trustedcoordinators/ | Trusted orchestration surfaces and internal coordinator functions | trustedcoordinatorfrontdoor / trustedcoordinatorinternal | Continue tightening scaffold, process chains, issue taxonomy, and caller migration | Medium | | runtime/storage/ | SQLite packet store, schemas, query services, low-level persistence | storageadapter | Keep concrete storage here; require Archive for coordinator/server reads where practical | Low | | runtime/nexus/server/packet-explorer/ | Explorer import/export/search/data payload services | runtimeadapter, projectioncandidate, compatibilitybridge | Keep API payloads stable while moving read models to Projection and packet movement to Exchange/Archive | Medium | | runtime/nexus/server/discussion/ | Discussion forums, posts, replies, and vote-adjacent projections | owaproductadapter, projectioncandidate, definitiondrivencandidate | Treat as transitional; move generic message/thread projection into Projection definitions over time | High | | runtime/nexus/server/reaction/ | Reaction/vote projection and reaction packet service logic | owaproductadapter, projectioncandidate, definitiondrivencandidate | Move target/reaction summaries and action availability toward Projection/Definition descriptors | High | | runtime/nexus/server/locality/ | Locality directory, graph planning, location search | owaproductadapter, definitiondrivencandidate, runtimeadapter | Keep OWA locality conventions explicit; move reusable graph/build/dependency rules to Planning/Definition where possible | High | | runtime/nexus/server/scope/ | Scope graph, ancestry compatibility, parent resolution, display preferences | owaproductadapter, projectioncandidate, compatibilitybridge | Separate generic graph projection from OWA policy/profile adapters and legacy compatibility reads | High | | runtime/nexus/server/identity/ | Sessions, passkeys, actor custody, auth storage, identity search | runtimeadapter, storageadapter, compatibilitybridge | Identity custody remains runtime-owned; packet-specific read/display pieces can move toward Projection | Medium | | runtime/nexus/server/readiness/ | Modernization and pre-reseed readiness reports | testoraudit | Keep as audit/report layer; update as seams migrate | Low | | remaining mutation- / packet-runtime-* ledgers | Static mutation intent descriptors, ticket compatibility helpers, handoff/readiness ledgers | compatibilitybridge, testoraudit, legacysignedcorridor | Keep only what describes transitional state; route executor files are removed | Medium | | top-level nexus-query-data.ts | Broad Nexus read model aggregation | projectioncandidate, owaproductadapter | Decompose by projection responsibility; move generic packet/card/list/read models behind Projection | High | | top-level verification-service.ts | Verification wrapper, validator identity, report writing | compatibilitybridge, runtimeadapter | Keep wrapper short; move report certification/storage decisions later through Certification/Archive | Medium |

Coordinator readiness and gaps

The major trusted coordinator doors are present: Dispatch, Definition, Regulation, Planning, Building, Inspection, Certification, Archive, Compatibility, Verification, Exchange, and Projection. There is intentionally no separate Write Coordinator; write lifecycle orchestration belongs to Dispatch. The remaining work is caller migration and internal cleanup, not inventing another major coordinator.

Coordinator gaps to address:

| Gap | Current signal | Ideal result | Recommended pass | | --- | --- | --- | --- | | Resolution is legacy-flat | scaffold audit warning | folder only if it becomes a central authority seam | Coordinator loose ends | | Packet workflow and composite workflow are transitional | large flat files with product-aware operation branches | either folder as transitional coordinators or retire into Planning/Building/Inspection flows | Coordinator loose ends | | Projection is under-adopted | read models still live in server services | Projection owns generic packet/list/graph/read-model output with adapters preserving route payloads | Projection migration | | Regulation remains light | policy/write-gate meaning still partly server-owned | Regulation owns reusable policy and write-gate resolution; OWA policies remain adapters/profile data | Regulation deepening | | Exchange/Archive/Verification are partially adopted | Explorer paths improved, but server callers still have direct storage/signature/interpreter touches | callers ask Exchange, Archive, Verification, and Compatibility first | Crossing migration |

Legacy signed corridor inventory

Dispatch is the route-facing write lifecycle owner for /api/nexus/mutations/prepare and /api/nexus/mutations/finalize. New architecture language should describe enrolled writes as a Dispatch-owned pipeline through Definition, Regulation, Planning, Building, Inspection, Certification, Verification, and Archive.

| Process | Current files | Ideal owner | Ideal location | Risk | | --- | --- | --- | --- | --- | | mutation prepare/finalize orchestration | trusteddispatchcoordinator/ plus thin mutation API routes | Dispatch plus Regulation/Planning/Certification/Verification/Archive handoffs | relation.follow.add, relation.association.add, and reaction.vote.set complete the Dispatch-owned route chain; other intents remain capability gaps, not fallback paths | High | | proof/ticket storage | prepared-write-ticket-service.ts, prepared-write-ticket-store.ts | Certification owns signature handoff; Verification owns signed packet validation; Archive owns storage | keep ticket helpers only as transitional compatibility until Certification ticket durability fully replaces them | Medium | | intent schema and enrollment | prepare-mutation-intent-schema.ts, mutation-intent-registry.ts, packet-client-intent-enrollment.ts | Dispatch intake plus Definition-driven client/action metadata | Dispatch/Definition with API schema bridge | Medium | | domain-specific finalize handlers | removed legacy handler-domain executor files | Planning/Building/Inspection/Archive or OWA adapter when product-specific | migrate remaining product intents directly into coordinator chains or adapter bridges, not old handler maps | High | | genericization/handoff audits | packet-runtime-dispatch-handoff.ts, workflow alignment audits | readiness/test audit | readiness/ after terminology cleanup | Low | | Preference write workflow | preference-runtime-connectors.ts, preference runtime connector files | Definition/Planning/Archive plus compatibility cache adapter | migrate claimed Preference.element writes through the trusted chain and retire compatibility cache writes when safe | Medium |

Packet-type-specific service inventory

These areas are not wrong for the current prototype, but they are the main distance from the ideal Definition-driven runtime.

| Process | Current behavior | Ideal form | Migration risk | | --- | --- | --- | --- | | Discussion projections | hardcoded forum/topic/post/message/root/reply interpretation | Definition and Projection descriptors produce message/thread/read models; OWA adapter preserves route payloads | High | | Reaction/vote projections | hardcoded target, actor, vote, association, and support summaries | Projection/action descriptors produce target reaction panes and available actions | High | | Locality graph planning | OWA locality hierarchy, descriptor, parent, and graph rules in service code | OWA adapter plus Definition/Planning dependency descriptors for reusable graph build rules | High | | Scope graph | canonical relations plus legacy claim/home compatibility and OWA policy anchors | Projection graph base plus OWA scope profile adapter and explicit compatibility bridge | High | | Identity search and auth packet reads | Element.person checks and actor packet assumptions | identity custody stays runtime-owned; display/search projections move toward Projection descriptors | Medium | | Packet Explorer data panels | generic inspector and search card rows now use Projection/Archive seams; Export/Import retain service-specific logic | continue migrating remaining Explorer helper paths behind coordinator seams when response parity is straightforward | Medium | | Query data aggregation | one broad route-read model service mixes many packet semantics | split into projection-backed read models plus product-specific adapters | High | | Verification reports | service writes Report packets directly after coordinator assessment | Certification/Archive/report handoff decides durable signed diagnostics | Medium |

Packet-definition opportunities

Move repeated hardcoded behavior into packet definitions only when the definition describes capability or display metadata, not executable untrusted code.

Good candidates:

- projection field labels, badges, summary fields, card sections, and preferred surfaces
- action availability descriptors and action grouping hints
- packet subtype display rules and generic target/source ref labeling
- default/dependency/build descriptors that Planning can compile into operation plans
- policy requirement descriptors that Regulation can evaluate with trusted local code
- import/export/verification summary descriptors for display and review

Not candidates for imported executable definition logic:

- SQLite reads/writes
- private-key custody, proof checks, and session validation
- signature verification implementation
- policy enforcement execution
- adapter code for schema migration
- API route authorization

Ideal destination map

| Current pressure area | Ideal destination | First migration step | | --- | --- | --- | | nexus-query-data.ts | Projection coordinator plus OWA read adapter | carve out one route payload behind Projection with exact response parity | | discussion service | Projection definitions plus OWA discussion adapter | isolate projection-only helpers from mutation/session behavior | | reaction service | Projection/action descriptors plus OWA reaction adapter | separate target reaction read model from write path | | locality directory | OWA locality adapter plus Planning/Definition descriptors | identify generic graph/dependency planning pieces | | scope graph | Projection graph base plus OWA scope adapter | split legacy compatibility reads from canonical graph assembly | | Packet Explorer data | Projection/Archive/Compatibility bridge | continue moving Export/Import edge cases behind coordinator seams after the generic inspector and search-card projection migrations | | legacy mutation corridor remnants | Dispatch-owned coordinator chain | keep deleted executor guardrails in audit; classify remaining ticket/handoff ledgers before removal | | verification service | Verification wrapper plus report Certification/Archive handoff | keep assessment wrapper, isolate report-writing surface |

Running cleanup tab

Use this as the short check-off ledger so the same issues do not have to be rediscovered by later audits.

| Item | Status | Notes | | --- | --- | --- | | Trusted coordinator scaffold/test scripts | Done | audit:trusted-coordinators, test:trusted-coordinators, and check:trusted-coordinators are package-level entrypoints. | | Empty Write Coordinator remnant | Done | Removed; writes remain Dispatch-owned. | | Exchange commit accepted-entry narrowing | Done | Commit bundles are narrowed to accepted plan entries and compare Archive import receipts against the Exchange plan. | | Dispatch finalize raw signed-material handoff | Done | Finalize routes no longer parse signed packets before Dispatch; Verification receives raw signed packet material. | | Certification/Verification/Archive key continuity | Done | Certification records certified packet revision keys; Dispatch compares Certification against Verification and Archive; Archive blocks mismatched extracted envelopes. | | Mutation intent label authority | Done | Dispatch derives finalized kind from the certified plan and rejects mismatched caller labels. | | Resolution foldering | Open | Still the expected legacy-flat warning in the trusted coordinator scaffold audit. | | Direct storage/signature/interpreter runtime crossings | Open | Packet Explorer generic inspector no longer uses the legacy packet interpreter. Direct storage touches are now explicitly classified; remaining coordinator-migration targets are discussion writes, reaction writes, Preference.element helper writes, and Packet Explorer import preferred-head repair. | | Packet Explorer generic projection adoption | Done | The generic inspector payload now gets revision state, edges, raw/adapted reads, and read-model projection through Trusted Archive/Projection while preserving API response shape. | | Packet Explorer search card projection | Done | Search ranking/grouping remains service-owned, but selected search rows now pass through Trusted Projection card-list output before mapping back to the existing response shape. | | Generic query card projection | Partial | Dashboard, votes, and library packet-card lists now pass already-selected cards through Trusted Projection; discussion/reaction/scope/locality read models remain adapter migration lanes. | | Projection adoption | Open | Packet Explorer Export/Import edges, deeper nexus-query-data, discussion/reaction/scope read models remain the biggest migration lane. | | Legacy mutation service registry dependency | Done | NexusMutationService is no longer constructed or exposed by the live Nexus packet service registry. | | Reaction finalize derived-state bridge | Done | reaction.vote.set derived-state response decoration moved out of Trusted Dispatch and into the reaction runtime adapter used by the finalize route. | | Legacy signed-corridor executor removal | Done | Removed the old NexusMutationService, signed-packet finalizer, Dispatch prepare/finalize handlers, old handler domain maps, manifest Dispatch metadata bridges, and preference Dispatch workflow. Runtime/UI naming now uses Dispatch-owned corridor language. | | OWA adapter/profile split | Open | Discussion, reaction, locality, and scope still mix generic runtime with OWA product behavior. |

Recommended migration order

1. Keep route-facing write lifecycle under Dispatch. Removed legacy executor files must stay deleted; remaining handoff/readiness ledgers should stay Dispatch-named and descriptor-only. 2. Move read-model work toward Projection Coordinator where payload parity is straightforward: Packet Explorer's generic inspector and search-card paths have started this migration, and generic dashboard/votes/library cards are partially projected; continue with deeper route query data, then discussion/reaction/scope read models. 3. Move the direct storage migration-target bucket behind Archive, Exchange, Verification, Projection, and Compatibility. The classification guard is in place; next cleanup should reduce the needstrustedcoordinator bucket rather than rediscovering it. 4. Extend the proven Dispatch write pipeline beyond the first migrated intents: full packet envelope materialization, Certification signed-bundle checks, Verification handoff, Archive storage, and result projection for each live intent. Reaction vote writes now use the chain, with derived summary/index refresh isolated in the finalize route's reaction adapter until Projection owns the read model. 5. Separate OWA-specific adapters from generic runtime. Product profile behavior should be named, not hidden inside generic services. 6. Retire compatibility bridges only after import scans and route tests prove callers have migrated.

Acceptance standard for future cleanup

A runtime process is in its ideal form when:

- API routes are thin callers
- storage access goes through storage adapters or Archive-facing seams
- packet compatibility questions go through Compatibility
- import/export/merge movement goes through Exchange
- verification and signature assessment go through Verification or Certification as appropriate
- read models go through Projection unless they are explicitly product adapter output
- write flows enter through Dispatch and then move through Definition, Regulation, Planning, Building, Inspection, Certification, Verification, and Archive
- OWA-specific behavior is isolated as an adapter/profile layer
- packet definitions describe capabilities and display/build/policy metadata without executing untrusted code

Architecture And Layers

Architecture shape

The long-term architecture is a three-layer stack with strict boundaries:

Core Nexus / Core Contracts Vault

The portable packet law and machinery.

- packet schemas, definitions, builders, compatibility, and validation
- defaults definitions and dependencies definitions
- policy requirement contracts and deterministic graph logic
- canonicalization, signing, verification, import/export/merge rules
- no database ownership, private-key custody, route assumptions, UI state, or platform-specific behavior

Runtime / Trusted Runtime Coordinators

The trusted execution environment around Core.

- session and actor context
- request dispatch and validation
- policy/regulation resolution against live packet state
- packet planning, building, inspection, testing, certification, archival, verification, import, export, and projection
- storage adapters and API-facing glue
- signing-ticket flow and mutation finalization

Runtime coordinators feed live context through the Core Contracts Vault and execute trusted local code. Packet definitions can describe allowed behavior, but imported definitions do not execute runtime behavior.

Interface / Event Cockpit

The reusable user-facing cockpit.

- surfaces, page layout builders, cards, menus, tabs, badges, forms, modals, and loading overlays
- lightweight input validation and intent emission
- ticket/signing prompts, confirmations, errors, success states, and refresh requests
- projection consumption through view models

The interface should not decide packet validity, policy authority, or graph truth. The Interface Event Coordinator shapes user events, manages client-side validation/loading/refresh lifecycle, and submits intent toward the Trusted Dispatch Coordinator.

The load-bearing rule stays the same: data is the system; platforms are adapters.

Product framing

The current product stack remains:

- FCF = principles and coordination physics
- Nexus = portable signed-packet substrate and graph engine
- OWA = geography-first civic application profile on Nexus

OWA may provide default policies, default packets, projection preferences, governance presets, and seed choices. OWA should not mutate Nexus packet builders or redefine core packet anatomy.

Repository boundaries

The active repo split is:

- core/* for portable packet logic, schemas, builders, interpreters, definition contracts, pure projections, and graph rules
- runtime/* for trusted coordinators, persistence, runtime services, query services, auth/trust/discussion orchestration, and API glue
- app/* for Event Cockpit components, hooks, constants, public content, reusable UI, and shared state
- src/app/* for the Expo Router route shell and API entrypoints

The runtime folder is now organized around stable ownership zones:

- runtime/trusted_coordinators/* owns trusted orchestration front doors, result envelopes, process chains, and coordinator audits
- runtime/storage/* owns concrete persistence adapters and storage schemas
- runtime/nexus/server/* owns API-facing Nexus runtime services grouped by bounded context

The runtime/nexus/server bounded-context map is:

- identity/* for auth, passkeys, shell auth context, and identity search
- discussion/* for discussion projection and discussion surface helpers
- reaction/* for reaction and vote projection/services
- locality/* for locality directory, graph planning, and location search
- scope/* for scope graph, scope compatibility, parent resolution, and display preferences
- packet-explorer/* for Explorer import/export/search/data services
- dispatch/* or coordinator-owned folders for signed mutation prepare/finalize corridor work as later passes migrate it
- readiness/* for modernization and pre-reseed readiness reports
- shared/* for small server helpers used across bounded contexts

Top-level files in runtime/nexus/server may remain as temporary compatibility bridges during migration. New implementation code should prefer the bounded-context folders once a module has moved.

Documentation system

The multi-chapter internal docs now use a chapter-first source-of-truth model.

Canonical content lives in:

- docs/implementation-guide/*
- docs/specifications/*
- docs/roadmap/*

The top-level files:

- docs/implementation-guide.md
- docs/specifications.md
- docs/roadmap.md

remain short local index shells only.

Public docs generation is now expected to follow that structure:

- docs/public/public-docs.manifest.json owns the ordered public source lists
- scripts/validate-public-docs.mjs validates manifest and shell/source-of-truth rules
- scripts/build-public-docs.mjs compiles readable docs data, Markdown downloads, PDF downloads, and version records
- generated artifacts under app/public/generated/, public/downloads/, and docs/public/version-records/ are derived outputs and should not be edited by hand

Expected workflow:

- update chapter files first
- run npm run docs:validate after canon doc edits
- rely on npm run docs:build during site/export builds or when intentionally refreshing generated public docs artifacts

Core versus adapters

Core owns

- packet schemas and validation
- packet definitions, builders, defaults definitions, and dependencies definitions
- graph relationships and traversal

- import/export/merge rules
- signing and verification law
- policy requirement hooks and deterministic business logic

Runtime owns

- trusted coordinator workflows
- live packet lookup and storage writes
- actor/session context
- signing-ticket flow
- policy resolution against current state
- verification reports and runtime-owned indexes
- API-facing orchestration

Adapters and interface own

- routing and navigation
- rendering and layout
- input UX
- loading, confirmation, and error chrome
- device or browser integration
- projection display

Adapters should not invent parallel business logic for trust, validation, compatibility, policy, or merge behavior.

Nexus scoped loading

Nexus loading chrome is an app-layer concern. The scoped loading provider, boundary, overlay, and hook live under app/components/nexus/ui/feedback/loading/* and track caller-owned visual scopes such as a page, panel, tab, card, menu, or action pane.

The loading system must not depend on packet type, mutation intent, packet action registry entries, or runtime operation categories. Runtime and core code should remain unaware of blur overlays, spinners, and UI input blocking.

The active state starts immediately so duplicate input inside the same boundary can be blocked before the delayed visual overlay appears. The visible state is delayed to prevent spinner flicker on fast operations, and visible overlays may remain briefly to satisfy the minimum visible duration.

Shared Nexus buttons and action-menu items can carry optional loading scopes, allowing common action chrome to start scoped loading without knowing packet types or runtime operation semantics. The UI site still owns the visual scope choice because it knows which page, panel, tab, card, menu, or action pane should be blocked.

Navigation and shell model

The intended navigation model still uses three mental axes:

- place or scope
- topic or work
- time or evidence

The current implemented Nexus slice is a practical early version of that model, with:

- dashboard
- discussions
- votes
- roles
- trust
- library
- packet explorer as a shell-level inspection workspace
- account and identity custody as wrapper-level surfaces

Nexus UI Component Catalog

Purpose

This catalog records the current Nexus UI component landscape before a broader universal-template cleanup. It is an audit artifact only: it should guide later consolidation work without moving files, changing visuals, or coupling Nexus UI to the public site design system.

Public-site components may be useful references, but Nexus should keep its own component system because the public site and Nexus workspace are separate product surfaces.

For exact file-level counts, route-local component candidates, raw React Native primitive usage, and extraction sequencing, see Nexus UI Hard Inventory.

Current pressure points

- src/app/nexus/locality/create.tsx and src/app/nexus/discussions.tsx are the largest route-local UI surfaces and contain many one-off controls, pickers, composers, modals, and layout sections.
- app/components/nexus/nexus-ui.tsx is now a temporary compatibility bridge; shared primitive implementations live in focused app/components/nexus/ui/* component-type modules.
- app/components/nexus/ui/tabs/nexus-tabs.tsx, app/components/nexus/ui/tabs/nexus-tab-primitives.tsx, and app/components/nexus/features/explorer/nexus-packet-explorer-tab-deck.tsx already share some tab skeleton pieces, but the Explorer tab deck remains a separate document-tab system.
- Modal and overlay behavior is spread across route-local Modal usage, auth gates, feature-status explainers, packet explorer overlays, sidebar drawers, and loading boundaries.
- Action chrome is the most mature shared area: packet cards, badge strips, menu buttons, action menus, action lists, and scoped loading plumbing already form a reusable foundation.

Folder foundation status

The first Nexus UI folder foundation pass moved stable shared modules into app/components/nexus/ui/* without changing visuals or route structure:

- loading/ now lives at ui/feedback/loading/
- action-card/ now lives at ui/cards/action-card/
- action-list/ now lives at ui/actions/action-list/
- nexus-tabs.tsx and nexus-tab-primitives.tsx now live under ui/tabs/*

The older locations should not be reused for new shared UI. Larger feature folders and route-local surfaces are intentionally unchanged until later extraction passes.

Overlay consolidation status

The first universal-template consolidation pass added shared overlay primitives under app/components/nexus/ui/overlays/*:

- NexusModalShell owns the reusable Modal backdrop, close press target, centering, and Nexus card frame.
- NexusOutcomeDialog and NexusConfirmDialog compose common outcome and confirmation dialog patterns on top of the shell.
- NexusPopover reserves a lightweight shared frame for caller-positioned overlay panels.

The initial migration moved repeated modal chrome in dashboard packet validation, packet Explorer import outcomes, and locality create/reuse/picker dialogs onto the shared shell while preserving their existing visual classes and handlers. Auth/session logic remains outside the generic overlay primitives.

Primitive split status

The broad nexus-ui.tsx primitive file has been split into focused component-type modules while preserving visual behavior:

- card surfaces live in ui/cards/nexus-card.tsx
- action buttons and chevrons live in ui/actions/*
- badges live in ui/feedback/nexus-badge.tsx
- segmented controls and inline selects live in ui/forms/*
- chrome, appearance, bevel, and section-header helpers live in ui/layout/*
- attached tab rails live in ui/tabs/nexus-attached-tab-rail.tsx

Nexus callers should import from @app/components/nexus/ui or a direct family path. app/components/nexus/nexus-ui.tsx remains only as a temporary bridge for compatibility and should not be the source for new imports.

Primitive adoption checkpoint

The whole-Nexus primitive checkpoint classifies raw React Native usage into four buckets:

- official primitive internals, such as ui/* controls that still need Pressable, ScrollView, TextInput, Modal, Switch, or ActivityIndicator internally;
- acceptable feature-local controls, such as Explorer document tabs, sidebar rails, locality graph rows, discussion panels, and bounded modal result areas where behavior remains specialized;
- route/controller usage to migrate, where a route owns simple repeated shell chrome rather than behavior-specific controls;
- deferred behavior-sensitive usage, where scroll refs, keyboard focus, auto-load handlers, resize behavior, animation state, or shell state make a mechanical conversion unsafe.

As the first low-risk adoption cleanup, the trivial route-level ScrollView className="flex-1" showsVerticalScrollIndicator={false} shells in account, dashboard, discussions, roles, trust, and votes now use NexusScrollFrame. Scroll containers with refs, custom handlers, bounded modal bodies, graph focus behavior, Explorer workbench behavior, or sidebar shell animation remain intentionally local.

Current stale-import status: Nexus routes and feature components should not import primitives from @app/components/nexus/nexus-ui; that file remains only as the temporary compatibility bridge.

Pre-audit wrap-up status

Nexus now has the main feature-component homes expected by the consolidation roadmap:

- features/discussions/* for discussion workspace panels, cards, composers, vote controls, and reply trees.
- features/explorer/* for Packet Explorer shell, document tabs, search, import/export, validation, inspection, resize, and workbench composition.
- features/locality/* for locality create panels, graph rows, picker dialogs, outcome dialogs, and preview panels.
- features/sidebar/* for sidebar rail, menu, preference, scope row, and scope action composition.
- features/identity/* for identity route shells, fields, preference cards, route links, and location lookup UI.

Official reusable primitives live under ui/actions, ui/cards, ui/feedback, ui/forms, ui/forms/search, ui/layout, ui/overlays, and ui/tabs. Accepted exceptions remain where behavior is still specialized: Explorer document tabs, sidebar shell animation/drawer state, locality graph focus and picker orchestration, discussion auto-load scroll regions, auth gates, and identity route controller flows.

The remaining large post-audit targets are trust/roles feature extraction, a sidebar second split, dashboard/library/account/votes route polish, and any locality controller cleanup that does not disturb graph/search behavior. Interface Event Coordinator work is intentionally tracked in the runtime/interface orchestration chapter rather than this UI consolidation chapter.

Feature extraction status

The first large route decomposition passes created app/components/nexus/features/discussions/* for discussion-specific composed UI. The route still owns query normalization, data loading, mutations, auth gates, router navigation, and reply branch state, while extracted feature components render feed/thread/post workspace panels, feed post cards, root post cards, reply trees, vote/reply-count pills, and post/reply composers.

Discussion loading scopes are visual-boundary owned: feed load-more, root replies, reply branches, vote pills, and post/reply composers now have scoped loading seams without coupling loading to packet action registries or runtime mutation types.

The workspace panel shapes are intentionally feature-local but generic-friendly: feed panels, thread toolbars, recursive tree sections, and composer panels can be promoted into ui/* later if another Nexus surface proves the same reusable skeleton.

The sidebar has also started feature-local extraction under app/components/nexus/features/sidebar/*. nexus-sidebar.tsx remains the shell controller for router, auth, preference, rail-state, and scope mutation handling, while rail toggles, preference switches, function menus, scope rows, grouped scope sections, and scope action menus now live in the sidebar feature module.

Locality create has begun feature-local extraction under app/components/nexus/features/locality/*. The route still owns params, identity/auth gates, draft persistence, graph/path state, API calls, mutation handlers, and modal state, while search/build panels, picker dialogs, outcome dialogs, graph rows, and preview panels now live in locality feature components. The obsolete level-row ladder builder path was removed after extraction; the active create flow is graph-based.

Packet Explorer now lives under app/components/nexus/features/explorer/. The feature folder owns Explorer-specific shell, tab, toolbar, search, import, export, data, link, resize, and inspection panels, while shared ui/ primitives remain the source for generic cards, forms, search result chrome, feedback, loading, tabs, and workbench panel pieces. Export and import actions have caller-owned loading scopes for packet export, store export, import analysis, import commit, and import history. A later internal decomposition split import cards, export preview helpers, inspection subpanels, and the validation dialog into sibling Explorer feature files; packet validation also gained the visual scope packet-explorer:validation:<packetId>. Explorer document tabs remain feature-specific until a future tab-extension pass.

Identity route UI helpers now live under app/components/nexus/features/identity/*. Identity routes remain the controllers for router returns, auth/session state, passkey flows, storage decisions, mutations, and error state, while the feature folder owns route shells, field wrappers, preference cards, route links, and location lookup presentation/loading. app/components/nexus/nexus-identity-ui.tsx remains only as a temporary compatibility bridge.

Component Type Catalog

| Component type | Current files and usages | Current status | Duplication notes | Future template target | Migration risk | | --- | --- | --- | --- | --- | --- | | Cards and packet cards | ui/cards/nexus-card.tsx, ui/cards/action-card/, focus/, dashboard/, route-local nested cards in trust, roles, votes, library, discussions, locality/create, and packet explorer panels | Shared base plus feature-specific compositions | NexusCard is used widely, but route files still compose repeated metric cards, inset cards, warning cards, focused packet panels, and packet preview rows manually | ui/cards with base surface, metric/stat card, packet card, focused packet panel, warning/outcome card, and repeated inset section patterns | Medium | | Action buttons, menus, lists, and badge strips | NexusActionButton and NexusChevronIcon in ui/actions/; ui/cards/action-card/; ui/actions/action-list/; packet action conversion in packet-actions/; dashboard queue/list actions | Strongest shared foundation | Shared action menus and badge strips exist, but route-local action clusters and raw buttons still appear in large pages; button/menu scoped loading is now available but adoption is caller-owned | ui/actions with action button, icon/menu button, action menu, action cluster, action list, badge strip, and packet-action adapters | Low | | Tabs and tab decks | ui/tabs/nexus-tabs.tsx, ui/tabs/nexus-tab-primitives.tsx, route uses in roles, identity/sign-in, locality/create, packet explorer toolbar and primary rail; Explorer document tabs in features/explorer/nexus-packet-explorer-tab-deck.tsx | Shared navigation tabs plus separate Explorer document tabs | Function-surface tabs and Explorer tabs both use NexusTabFrame / NexusTabLabel, but differ in close controls, tooltip behavior, wrapping, and session/document semantics | ui/tabs with shared frame/label/close primitives, navigation rail/stack, segmented tabs, and Explorer document-tab extension | Medium | | Modals, gates, confirmations, and overlays | ui/overlays/, features/locality/, nexus-auth-gate.tsx, nexus-shell-entry-gate.tsx, nexus-feature-status-context.tsx, packet explorer shell overlay, remaining route-local overlay variants | Shared modal shell now exists; gates remain specialized | Dashboard validation, packet import outcome, and locality create/reuse/picker dialogs now use shared modal chrome; locality dialog content is feature-local while auth/entry gates and feature-status overlays still need careful migration because they carry session or shell-specific behavior | ui/overlays with modal shell, confirmation dialog, outcome dialog, anchored popover, blocking gate, and shell overlay host | Medium | | Dropdowns, selects, pickers, and menus | NexusInlineSelect in ui/forms/; ui/forms/search/; NexusActionMenu; locality kind/parent/result pickers; identity location search; packet explorer search/export lookup lists; sidebar menus | Mixed shared and route-local | Action menus and inline selects are shared; search dropdown/result chrome now has shared field/list/row primitives, while picker-specific side effects remain local | ui/menus or ui/forms with inline select, anchored menu, searchable result list, picker modal, and selectable row | Medium | | Text inputs, composers, search fields, and form rows | ui/forms/nexus-field-shell.tsx, ui/forms/nexus-text-input.tsx, ui/forms/nexus-text-area.tsx, ui/forms/nexus-field-action-row.tsx, ui/forms/search/, features/discussions/, features/identity/; raw TextInput remains in larger route-local specialized controls | Shared field/input/search chrome is now available | Identity fields now live in the identity feature folder and wrap generic shell/input primitives; Packet Explorer import/export fields, trust notes, roles support comments, and discussion post/reply composers use shared input/textarea seams. Broader route-local form sections still need careful migration | ui/forms with field shell, text input, textarea/composer, search box, result list, error text, hint text, and field action row | Medium | | Loading and feedback states | ui/feedback/loading/; ui/feedback/nexus-feedback-states.tsx; discussion and locality feature loading scopes; local isLoading flags in route pages; warnings/errors with NexusCard tones | Shared loading and basic feedback cards now exist | Scoped loading provider/boundary exists;

Route-local areas to audit first during consolidation

1. src/app/nexus/locality/create.tsx

   - Remains the controller for params, auth gates, draft state, graph/path derivation, API calls, mutations, and handler ownership.

   - First feature extraction now lives in features/locality/*: search/build panels, picker dialogs, selected-result/remove/outcome dialogs, graph rows, preview panels, and shared locality-create types. The legacy level-row builder was removed; remaining cleanup should target repeated picker/list skeletons only after another surface proves reuse.

2. src/app/nexus/discussions.tsx

   - Still owns feed/thread/post workspace state, routing, loading, mutations, auth gates, reply branch state, and pagination actions.

   - Feature extraction now lives in features/discussions/*: feed/thread/post panels, feed cards, root post cards, vote/reply-count pills, reply tree controls, and post/reply composers. Future work can consider promoting workspace panel shells, scrollable feed panels, thread toolbars, composer shells, reaction/vote skeletons, and recursive tree rails only after non-discussion reuse appears.

3. app/components/nexus/nexus-sidebar.tsx

   - Still owns shell state, router/auth hooks, preference persistence, rail animation, and scope mutation handlers.

   - First feature extraction now lives in features/sidebar/*: rail toggles, guest avatar, preference switch, current context card, function menu rows, scope section headers, grouped scope rows, scope action menus, and scope menu content. Future work can promote rail toggles, compact nav rows, grouped collapsible sections, and anchored compact action menus only after other surfaces need the same shape.

4. app/components/nexus/features/explorer/*

   - Contains Explorer-specific tabs, toolbars, import/export/search panels, data panels, link groups, validation dialog, import/export cards, and packet inspection panels.

   - Next extraction targets: document-tab extension, shared workbench result/feed shell, lookup/result list promotion, and packet inspection card patterns if another surface proves reuse.

5. Identity routes and features/identity/*

   - Identity route shells, field wrappers, preference cards, route links, and location lookup UI now live in the identity feature folder, while auth/session/passkey/storage behavior remains route-owned.

   - Remaining post-audit targets are selectable identity rows/cards, security passkey/session cards, and preference panels if another surface proves the same reusable skeleton.

6. Primitive-adoption cleanup queue

- Remaining large extraction targets are locality create controller cleanup and possible follow-up feature splitting, identity sign-in/security/create/claim wrappers, trust and roles route-local form/panel sections, sidebar second split, and dashboard/library/votes/account route-shell cleanup.
- Loading adoption should continue only where the route or feature component owns both the async work and the visual boundary. The scoped foundation is already in place for discussions, locality, sidebar scope actions, search result regions, and Explorer import/export/validation surfaces.

Recommended target organization

Future consolidation should create a Nexus-native shared UI home without moving everything at once:

- app/components/nexus/ui/actions
- action buttons, icon/menu buttons, action menus, action clusters, action lists, badge strips
- app/components/nexus/ui/cards
- base cards, packet cards, metric/stat cards, warning/outcome cards, focus/preview cards
- app/components/nexus/ui/tabs
- tab primitives, navigation rails/stacks, segmented controls, Explorer document-tab extensions
- app/components/nexus/ui/overlays
- modal shell, confirmation dialog, outcome dialog, popover, feature-status overlay, auth/entry gates
- app/components/nexus/ui/forms
- field shell, text input, textarea/composer, action row, search/*, picker shell, toggles
- app/components/nexus/ui/layout
- page frame, scroll frame, section header, panel/workbench panel, section band, toolbar row, metric grid, rail section, panel split, responsive content frame
- app/components/nexus/ui/feedback
- loading boundary/overlay exports, empty state, error state, warning state, status rows

The existing folders can then become either adapters around the shared UI or feature-specific compositions:

- features/explorer/* should keep Explorer state/workbench logic but use shared tabs, overlays, forms, feedback, and panel frames.
- features/locality/* keeps locality graph/search semantics but uses shared picker, form/search, modal, loading, and warning/outcome components.

- features/discussions/* keeps discussion-specific workspace, post/reply/vote composition while consuming shared UI primitives and caller-owned loading scopes.
- features/sidebar/* keeps sidebar-specific rail, preference, function-menu, scope-list, and scope-action composition while consuming shared UI primitives and caller-owned loading scopes for async scope actions.
- features/identity/* keeps identity-specific route shells, form wrappers, location lookup presentation, preference cards, and route links while routes keep auth/session/passkey/storage behavior.
- ui/overlays/, ui/cards/action-card/, ui/actions/action-list/, ui/feedback/loading/, and ui/tabs/* now form the initial shared UI foundation.
- ui/forms/search/* now provides generic search field, result-list, result-row, status, empty/error, and loading-boundary wrappers. It intentionally does not own packet, identity, or locality search behavior.
- ui/forms/* now also provides generic field shell, text input, text area, and field action row primitives. Identity-specific field wrappers remain as compatibility adapters around those generic forms.
- ui/feedback/* now provides basic inline loading plus empty, error, warning, status, and operation card primitives. Scoped loading remains visual-scope owned by callers.
- ui/layout/* now provides page/scroll frames, panel/workbench wrappers, panel headers, section bands, toolbar rows, and metric grids. Panel loading remains caller-owned by visual scope.

Migration guidance

- Do not begin with a mass folder move. Promote one repeated pattern at a time.
- Preserve live visuals exactly unless a later task explicitly asks for design changes.
- Prefer extracting route-local UI only after at least two call sites need the same behavior.
- Keep runtime/core untouched; this is presentation infrastructure only.
- Treat public-site components as reference material, not dependencies.
- For each extraction pass, update this catalog or replace it with a more precise design-system chapter once the target folder structure becomes real.

Nexus UI Hard Inventory

Purpose

This chapter is the hard inventory behind the higher-level Nexus UI component catalog. It is a snapshot of the current Nexus UI landscape used to plan universal template extraction without changing live visuals or route behavior.

This is an audit artifact only. It should guide later consolidation, not imply that every item listed should move immediately.

Post-inventory folder foundation

After this snapshot, the first safe folder foundation pass moved stable shared modules into the canonical app/components/nexus/ui/* tree without visual or route behavior changes:

- app/components/nexus/loading/ -> app/components/nexus/ui/feedback/loading/
- app/components/nexus/action-card/ -> app/components/nexus/ui/cards/action-card/
- app/components/nexus/action-list/ -> app/components/nexus/ui/actions/action-list/
- app/components/nexus/nexus-tab-primitives.tsx -> app/components/nexus/ui/tabs/nexus-tab-primitives.tsx
- app/components/nexus/nexus-tabs.tsx -> app/components/nexus/ui/tabs/nexus-tabs.tsx

The first consolidation pass after the folder foundation added app/components/nexus/ui/overlays/* and moved repeated modal chrome in dashboard validation, packet Explorer import outcomes, and locality create/reuse/picker dialogs onto NexusModalShell while preserving existing visual classes and handlers.

The broad app/components/nexus/nexus-ui.tsx primitive file has since been split into focused ui/actions, ui/cards, ui/feedback, ui/forms, ui/layout, and ui/tabs modules. nexus-ui.tsx remains as a compatibility bridge only; new shared UI imports should use @app/components/nexus/ui or direct family paths.

The discussions route has since started its feature extraction into app/components/nexus/features/discussions/*. Feed/thread/post panels, feed/root post cards, vote/reply-count pills, recursive reply tree controls, and post/reply composers now live there, while src/app/nexus/discussions.tsx remains the route controller for query state, loading, mutations, auth gates, and reply branch state.

The sidebar has since started its feature extraction into app/components/nexus/features/sidebar/*. Rail toggles, guest avatar, preference switch, current context card, function menu content, scope section headers, grouped scope rows, scope action menus, and scope menu content now live there, while app/components/nexus/nexus-sidebar.tsx remains the shell controller for routing, auth gates, preference persistence, rail animation, and scope mutation handling.

Packet Explorer has since moved into app/components/nexus/features/explorer/*, including the shell overlay, document tabs, toolbar, search, import/export, data, link, resize, and inspection panels. The import/export panels now expose visual-scope loading seams for packet export, store export, import analysis, import commit, and import history, while Explorer tab/session behavior remains feature-local.

The Explorer feature folder has also had its first internal decomposition pass: import result/history cards, export preview/request helpers, packet inspection subpanels, and the validation dialog now live in sibling Explorer feature files. Packet validation uses the caller-owned visual scope packet-explorer:validation:<packetId>. The file-level counts below remain the original hard-inventory baseline rather than freshly recomputed post-decomposition counts.

The primitive-adoption checkpoint after that decomposition classified remaining raw primitive usage as official primitive internals, acceptable feature-local controls, route/controller usage to migrate, or deferred behavior-sensitive usage. The only mechanical route-shell conversions in that checkpoint were account, dashboard, discussions, roles, trust, and votes moving their trivial outer scroll wrappers to NexusScrollFrame. Library, locality create, Explorer, sidebar, and nested discussion panel scrolls remain deferred because they depend on refs, handlers, bounded regions, resize/session behavior, graph focus behavior, or shell animation.

The pre-audit wrap-up moved identity route UI helpers into app/components/nexus/features/identity/*. The feature folder owns identity page shells, field/input wrappers, preference cards, route-link cards, and location lookup presentation/loading. app/components/nexus/nexus-identity-ui.tsx remains a bridge only, and identity routes keep router, auth/session, passkey, storage, mutation, and error ownership.

The counts below remain useful as the pre-move hard inventory. New shared UI should prefer the ui/* paths.

Snapshot summary

- Scope inspected: app/components/nexus/ and src/app/nexus/.
- Files inspected: 83 non-test TypeScript/TSX files.
- Total lines inspected: 29662.
- Zones: feature 30, mixed 9, route 15, shared 19, support 10.
- Raw React Native primitive references: Pressable 158, ScrollView 62, TextInput 32, Modal 30, Switch 3, ActivityIndicator 2.
- Shared Nexus primitive references: NexusCard 358, NexusBadge 201, NexusActionButton 187, NexusSectionHeader 19, NexusSegmentedPill 17, NexusPreviewPanel 14, NexusTabFrame 12, NexusTabRail 12, NexusTabLabel 8, NexusActionList 7, NexusActionListItem 5, NexusInlineSelect 5, NexusActionMenu 4, NexusFocusedPacketSection 4, NexusTabStack 4, NexusActionCard 2, NexusLoadingBoundary 2.

Largest UI pressure files

| Lines | Zone | File | Primary component candidates | Raw primitives | | ---: | --- | --- | --- | --- | --- | | 2470 after feature extraction and cleanup | route | src/app/nexus/locality/create.tsx | NexusLocalityCreatePage controller | Pressable 0, direct modal chrome moved to features/locality/*, TextInput refs retained, ScrollView 4 | | 2447 | route | src/app/nexus/discussions.tsx | InlineReplyComposer, DiscussionVotePill, ReplyCountPill, ReplyNode, ReplyTree, NexusDiscussionsPage | Pressable 9, TextInput 4, ScrollView 7 | | 2056 | support | app/components/nexus/identity-shell-context.tsx | IdentityShellContext, IdentityShellProvider | - | | 1962 | mixed | app/components/nexus/nexus-sidebar.tsx | NexusGuestAvatar, NexusRailToggle, NexusPreferenceSwitch, NexusCurrentContextCard, NexusMenuSectionLabel, NexusPrimaryNavItem | Pressable 26, ScrollView 6, Switch 2 | | 1261 | shared | app/components/nexus/nexus-ui.tsx | NexusThemedBevelEdges, NexusBevelEdges, NexusCard, NexusSectionHeader, NexusBadge, NexusChevronIcon | Pressable 18, Modal 3, ScrollView 3 | | 967 | feature | app/components/nexus/features/explorer/nexus-packet-explorer-import-panel.tsx | ImportResultCard, ImportHistoryCard, NexusPacketExplorerImportPanel | Pressable 2, Modal 3, TextInput 2 | | 859 | support | app/components/nexus/nexus-shell-context.tsx | NexusShellContext, NexusShellProvider | - | | 795 | feature | app/components/nexus/features/explorer/nexus-packet-explorer-export-panel.tsx | ExportPreviewCard, NexusPacketExplorerExportPanel | Pressable 3, TextInput 6 | | 772 | mixed | app/components/nexus/features/explorer/nexus-packet-explorer.tsx | NexusPacketExplorer | Pressable 9, Modal 3 | | 768 | feature | app/components/nexus/features/explorer/nexus-packet-explorer-content.tsx | NexusPacketExplorerSeededSummary, NexusPacketExplorerLineagePanel, NexusPacketExplorerActionsPanel, NexusPacketExplorerValidationPanel, NexusPacketExplorerContent | ScrollView 3 | | 714 | route | src/app/nexus/roles.tsx | NexusRolesPage | TextInput 2, ScrollView 3 | | 676 | shared | app/components/nexus/nexus-tabs.tsx | NexusTabButton, NexusTabRail, NexusTabStack | ScrollView 5 | | 659 | route | src/app/nexus/trust.tsx | NexusTrustPage | TextInput 2, ScrollView 3 | | 655 | route | src/app/nexus/dashboard.tsx | NexusDashboardPage | Pressable 2, Modal 3, ScrollView 3 | | 639 | route | src/app/nexus/identity/sign-in.tsx | NexusIdentitySignInPage | Pressable 3 |

Raw primitive inventory

These are the files most likely to contain local button, modal, form, scroll, or overlay behavior that may eventually be promoted into Nexus-native templates. Counts are lexical references, not rendered instance counts.

| File | Pressable | Modal | TextInput | ScrollView | Other raw primitives | | --- | ---: | ---: | ---: | ---: | ---: | --- | | src/app/nexus/locality/create.tsx | 0 after feature extraction cleanup | 0 direct modal shells | 3 | 4 | legacy level-row builder removed | | app/components/nexus/nexus-sidebar.tsx | 26 | 0 | 0 | 6 | Switch 2 | | app/components/nexus/nexus-ui.tsx | 18 | 3 | 0 | 3 | - | | src/app/nexus/discussions.tsx | 9 | 0 | 4 | 7 | - | | app/components/nexus/features/explorer/nexus-packet-explorer.tsx | 9 | 3 | 0 | 0 | - | | app/components/nexus/features/locality/locality-create-graph-row.tsx | 9 | 0 | 3 | 0 | - | | app/components/nexus/action-card/nexus-card-badge-strip.tsx | 8 | 3 | 0 | 0 | - | | app/components/nexus/features/explorer/nexus-packet-explorer-export-panel.tsx | 3 | 0 | 6 | 0 | - | | app/components/nexus/nexus-identity-ui.tsx | 3 | 0 | 3 | 3 | - | | src/app/nexus/dashboard.tsx | 2 | 3 | 0 | 3 | - | | app/components/nexus/features/explorer/nexus-packet-explorer-import-panel.tsx | 2 | 3 | 2 | 0 | - | | app/components/nexus/nexus-tab-primitives.tsx | 7 | 0 | 0 | 0 | - | | app/components/nexus/preview/nexus-preview-panel.tsx | 3 | 0 | 0 | 3 | - | | src/app/nexus/roles.tsx | 0 | 0 | 2 | 3 | - | | app/components/nexus/nexus-tabs.tsx | 0 | 0 | 0 | 5 | - | | src/app/nexus/trust.tsx | 0 | 0 | 2 | 3 | - | | app/components/nexus/nexus-auth-gate.tsx | 4 | 0 | 0 | 0 | - | | src/app/nexus/library.tsx | 0 | 0 | 0 | 4 | - | | src/app/nexus/account.tsx | 0 | 0 | 0 | 3 | Switch 1 | | app/components/nexus/features/explorer/nexus-packet-explorer-content.tsx | 0 | 0 | 0 | 3 | - | | src/app/nexus/identity/sign-in.tsx | 3 | 0 | 0 | 0 | - | | app/components/nexus/features/locality/locality-create-preview-panel.tsx | 3 | 0 | 0 | 0 | - | | app/components/nexus/features/explorer/nexus-packet-explorer-tab-deck.tsx | 0 | 0 | 0 | 3 | - | | src/app/nexus/votes.tsx | 0 | 0 | 0 | 3 | - | | app/components/nexus/action-card/nexus-action-menu.tsx | 3 | 0 | 0 | 0 | - | | app/components/nexus/discussions/nexus-discussion-focus-panel.tsx | 3 | 0 | 0 | 0 | - | | app/components/nexus/action-list/nexus-action-list-item.tsx | 3 | 0 | 0 | 0 | - | | app/components/nexus/focus/nexus-focused-packet-section.tsx | 3 | 0 | 0 | 0 | - | | app/components/nexus/action-card/nexus-card-menu-button.tsx | 3 | 0 | 0 | 0 | - | | app/components/nexus/features/explorer/nexus-packet-explorer-search-panel.tsx | 0 | 0 | 2 | 0 | - | | app/components/nexus/nexus-shell.tsx | 2 | 0 | 0 | 0 | - | | app/components/nexus/nexus-feature-status-context.tsx | 2 | 0 | 0 | 0 | - | | app/components/nexus/nexus-shell-entry-gate.tsx | 2 | 0 | 0 | 0 | - | | app/components/nexus/loading/nexus-loading-overlay.tsx | 0 | 0 | 0 | 0 | ActivityIndicator 2 | | app/components/nexus/loading/nexus-loading-boundary.tsx | 2 | 0 | 0 | 0 | - |

Shared component inventory

These files already behave like shared UI substrate or close relatives. They should generally be reorganized before route files are rewritten.

| File | Lines | Exported/public components | Local component helpers | Families | Shared primitives referenced | | --- | ---: | --- | --- | --- | --- | | app/components/nexus/action-card/nexus-action-card.tsx | 81 | NexusActionCard | - | actions, cards | NexusActionCard 1, NexusCard 4 | | app/components/nexus/action-card/nexus-action-menu-controller.tsx | 159 | NexusActionMenuControllerProvider | NexusActionMenuControllerContext | actions, cards | NexusActionMenu 2 | | app/components/nexus/action-card/nexus-action-menu.tsx | 139 | NexusActionMenu | - | actions, cards | NexusActionMenu 1 | | app/components/nexus/action-card/nexus-card-action-cluster.tsx | 137 | NexusCardActionCluster | - | actions, cards | - | | app/components/nexus/action-card/nexus-card-badge-strip.tsx | 484 | NexusCardBadgeStrip | NexusCardBadgeIconView, NexusCardBadgeButton | actions, cards, overlays | - | | app/components/nexus/action-card/nexus-card-menu-button.tsx | 57 | NexusCardMenuButton | - | actions, cards | - | | app/components/nexus/action-list/nexus-action-list-item.tsx | 108 | NexusActionListItem | - | actions | NexusActionListItem 1 | | app/components/nexus/action-list/nexus-action-list.tsx | 40 | NexusActionList | - | actions | NexusActionList 1 | | app/components/nexus/loading/nexus-loading-boundary.tsx | 52 | NexusLoadingBoundary | - | feedback | NexusLoadingBoundary 1 | | app/components/nexus/loading/nexus-loading-context.tsx | 341 | NexusLoadingProvider | NexusLoadingContext | cards, feedback | - | | app/components/nexus/loading/nexus-loading-overlay.tsx | 56 | NexusLoadingOverlay | - | overlays, feedback | - | | app/components/nexus/nexus-tab-primitives.tsx | 427 | NexusTabFrame, NexusTabLabel, NexusTabDetail, NexusTabCloseButton | NexusTabBevelEdges | actions, tabs | NexusTabFrame 1, NexusTabLabel 1 | | app/components/nexus/nexus-tabs.tsx | 676 | NexusTabButton, NexusTabRail, NexusTabStack | - | actions, tabs, layout | NexusTabFrame 3, NexusTabLabel 2, NexusTabRail 2, NexusTabStack 2 | | app/components/nexus/nexus-ui.tsx | 1261 | NexusThemedBevelEdges, NexusBevelEdges, NexusCard, NexusSectionHeader, NexusBadge, NexusChevronIcon, NexusActionButton, NexusSegmentedPill, NexusAttachedTabRail, NexusInlineSelect | - | actions, cards, tabs, overlays, forms, layout | NexusActionButton 1, NexusCard 1, NexusSectionHeader 1, NexusInlineSelect 1, NexusSegmentedPill 1, NexusTabFrame 3, NexusTabLabel 2, NexusBadge 1 |

Feature component inventory

These files are feature-specific compositions. Most should keep their domain/controller logic but gradually consume shared ui/* templates for chrome, forms, overlays, panels, and feedback.

| File | Lines | Component candidates | Families | Raw primitives | Shared primitives referenced | | --- | ---: | --- | --- | --- | --- | | app/components/nexus/features/discussions/ | post-inventory extraction | DiscussionFeedPanel, DiscussionThreadPanel, DiscussionPostPanel, DiscussionThreadToolbar, DiscussionFeedPostCard, DiscussionRootPostCard, DiscussionReplyTree, DiscussionVotePill, DiscussionReplyCountPill, DiscussionPostComposer, DiscussionReplyComposer | cards, actions, forms, feedback, layout | Pressable retained only inside feature controls | NexusActionButton, NexusCard, NexusBadge, NexusLoadingBoundary, NexusTextInput, NexusTextArea | | app/components/nexus/discussions/nexus-discussion-focus-panel.tsx | 127 | NexusDiscussionFocusPanel | cards, layout | Pressable 3 | NexusActionButton 2, NexusCard 3, NexusBadge 3 | | app/components/nexus/focus/nexus-focused-packet-section.tsx | 105 | NexusFocusedPacketSection | cards, layout | Pressable 3 | NexusFocusedPacketSection 1 | | app/components/nexus/features/sidebar/ | post-inventory extraction | NexusRailToggle, NexusGuestAvatar, NexusPreferenceSwitch, NexusCurrentContextCard, NexusFunctionMenuContent, NexusScopeActionMenu, NexusScopeListRow, NexusGroupedScopeRows, NexusScopeMenuContent | actions, cards, forms, layout, feedback | Pressable and ScrollView retained inside feature controls | NexusCard, NexusCardMenuButton, NexusChevronIcon, NexusLoadingBoundary, NexusThemedBevelEdges | | app/components/nexus/features/locality/ | post-inventory extraction | LocalityCreateSearchPanel, LocalityCreateBuilderPanel, LocalityParentPickerDialog, LocalityKindPickerDialog, LocalityCreateKindDialog, LocalitySelectedResultDialog, LocalityRemoveConfirmDialog, LocalityOutcomeDialogs, LocalityCreateGraphRow, LocalityCreatePreviewPanel | overlays, forms, cards, layout, feedback | Pressable/TextInput/ScrollView retained inside feature controls | NexusActionButton, NexusBadge, NexusCard, NexusLoadingBoundary, NexusModalShell, NexusSearchField, NexusSearchResultList | | app/components/nexus/nexus-auth-gate.tsx | 566 | NexusAuthGateModal | overlays | Pressable 4 | NexusActionButton 3, NexusCard 3 | | app/components/nexus/features/identity/ | post-inventory extraction | IdentityPageShell, IdentityField, IdentityInput, IdentityRouteLinks, IdentityPreferenceCard, LocationLookupField | cards, forms, layout, feedback | Search/result loading is feature-local; route auth/session controls remain route-owned | NexusActionButton, NexusCard, NexusSectionHeader, NexusSegmentedPill, NexusSearchField | | app/components/nexus/features/explorer/nexus-packet-explorer.tsx | 772 | NexusPacketExplorer | overlays | Pressable 9, Modal 3 | NexusActionButton 2, NexusCard 3 | | app/components/nexus/nexus-shell-entry-gate.tsx | 113 | NexusShellEntryGate | overlays, layout | Pressable 2 | NexusActionButton 2, NexusCard 3, NexusBadge 2 | | app/components/nexus/nexus-shell.tsx | 243 | NexusShell | layout | Pressable 2 | - | | app/components/nexus/nexus-sidebar.tsx | 881 after feature extraction | NexusSidebar controller; rail/menu/scope/preference components moved to features/sidebar/* | actions, cards, forms, layout, feedback | route still owns profile/prefs drawer shell Pressables and rail ScrollViews | NexusCard, NexusSegmentedPill, NexusThemedBevelEdges | | app/components/nexus/features/explorer/nexus-packet-explorer-content.tsx | 768 | NexusPacketExplorerSeededSummary, NexusPacketExplorerLineagePanel, NexusPacketExplorerActionsPanel, NexusPacketExplorerValidationPanel, NexusPacketExplorerContent | actions, layout | ScrollView 3 | NexusActionButton 6, NexusCard 29, NexusBadge 11 | | app/components/nexus/features/explorer/nexus-packet-explorer-data-panel.tsx | 210 | NexusPacketExplorerDataPanel | layout | - | NexusCard 11, NexusBadge 7 | | app/components/nexus/features/explorer/nexus-packet-explorer-export-panel.tsx | 795 | ExportPreviewCard, NexusPacketExplorerExportPanel | cards, forms, layout | Pressable 3,

Route-local component inventory

Route-local components are the highest-risk extraction zone because they often blend controller state, data fetching, and UI chrome. They should be audited for behavior before any extraction.

| Route file | Lines | Local component candidates | Families | Raw primitives | Shared primitives referenced | | --- | ---: | --- | --- | --- | --- | | src/app/nexus/_layout.tsx | 71 | NexusLayoutContent, NexusLayout | layout | - | - | | src/app/nexus/account.tsx | 203 | NexusAccountPage | forms | Outer route shell now uses NexusScrollFrame, Switch 1 | NexusActionButton 7, NexusCard 9, NexusSectionHeader 2, NexusBadge 9 | | src/app/nexus/dashboard.tsx | 655 | NexusDashboardPage | overlays | Pressable 2, Modal 3, outer route shell now uses NexusScrollFrame | NexusActionButton 3, NexusCard 11, NexusSectionHeader 2, NexusFocusedPacketSection 2, NexusPreviewPanel 11, NexusActionList 5, NexusActionListItem 3, NexusBadge 5 | | src/app/nexus/discussions.tsx | 1691 after panel extraction | NexusDiscussionsPage route/controller; workspace panels and post/reply/vote feature components moved to features/discussions/* | forms, cards, feedback, layout | outer route shell now uses NexusScrollFrame; behavior-specific panel scrolls remain feature-local | NexusActionButton, NexusCard, NexusSectionHeader, NexusTabStack, NexusBadge | | src/app/nexus/identity/claim.tsx | 425 | NexusIdentityClaimPage route/controller | support | imports identity feature UI; route keeps auth/session/storage/mutation state | NexusActionButton 8, NexusCard 9, NexusBadge 3 | | src/app/nexus/identity/create.tsx | 328 | NexusIdentityCreatePage route/controller | support | imports identity feature UI; route keeps auth/session/storage/mutation state | NexusActionButton 5, NexusCard 3 | | src/app/nexus/identity/restore.tsx | 152 | NexusIdentityRestorePage route/controller | support | imports identity feature UI; route keeps restore/import handlers | NexusActionButton 5, NexusCard 3 | | src/app/nexus/identity/security.tsx | 588 | NexusIdentitySecurityPage route/controller | support | imports identity feature UI; route keeps passkey/session/security handlers | NexusActionButton 12, NexusCard 25, NexusSegmentedPill 3, NexusBadge 11 | | src/app/nexus/identity/sign-in.tsx | 639 | NexusIdentitySignInPage route/controller | support | imports identity feature UI; route keeps search/sign-in/passkey/import handlers | NexusActionButton 7, NexusCard 13, NexusTabRail 2 | | src/app/nexus/index.tsx | 14 | NexusIndexPage | support | - | - | | src/app/nexus/library.tsx | 317 | NexusLibraryPage | support | ScrollView 4 | NexusActionButton 5, NexusCard 9, NexusSectionHeader 2, NexusBadge 5 | | src/app/nexus/locality/create.tsx | 2470 after feature extraction cleanup | NexusLocalityCreatePage route/controller; search/build panels, picker dialogs, graph rows, preview panels, and obsolete level-row UI moved or removed | overlays, forms | route owns graph state, loading, mutations, auth, and outer scroll composition | NexusCard, NexusSectionHeader, NexusTabRail, NexusBadge | | src/app/nexus/roles.tsx | 714 | NexusRolesPage | forms | TextInput 2, outer route shell now uses NexusScrollFrame | NexusActionButton 7, NexusCard 23, NexusSectionHeader 2, NexusTabRail 2, NexusBadge 17 | | src/app/nexus/trust.tsx | 659 | NexusTrustPage | forms | TextInput 2, outer route shell now uses NexusScrollFrame | NexusActionButton 9, NexusCard 31, NexusSectionHeader 2, NexusBadge 20 | | src/app/nexus/votes.tsx | 210 | NexusVotesPage | support | Outer route shell now uses NexusScrollFrame | NexusActionButton 4, NexusCard 13, NexusSectionHeader 2, NexusBadge 7 |

Support and context files with UI adjacency

These are not primary UI templates, but several contain providers, controllers, or state that UI templates depend on. They should remain outside presentation folders unless their visual pieces are separated first.

| File | Lines | Component/provider candidates | Exported types | Zone | | --- | ---: | --- | --- | --- | | app/components/nexus/identity-shell-context.tsx | 2056 | IdentityShellContext, IdentityShellProvider | - | support | | app/components/nexus/identity-shell-dispatch-adapter.ts | 367 | - | - | support | | app/components/nexus/nexus-auth-gate-types.ts | 12 | - | - | support | | app/components/nexus/nexus-feature-status-context.tsx | 234 | NexusFeatureStatusContext, NexusFeatureStatusOverlay, NexusFeatureStatusProvider | - | support | | app/components/nexus/nexus-feature-status-registry.ts | 203 | - | NexusFeatureStatusKind, NexusFeatureStatusEntry, NexusFeatureStatusId | support | | app/components/nexus/nexus-route-utils.ts | 77 | - | - | support | | app/components/nexus/nexus-shell-chrome-context.tsx | 56 | NexusShellChromeContext, NexusShellChromeProvider | - | support | | app/components/nexus/nexus-shell-context.tsx | 859 | NexusShellContext, NexusShellProvider | - | support | | app/components/nexus/nexus-shell-preferences.ts | 67 | - | NexusShellPreferenceMode, PersistNexusElementPreferenceInput, PersistNexusElementPreferenceResult | support | | app/components/nexus/packet-actions/nexus-packet-action-registry.ts | 57 | - | NexusPacketActionInput, NexusPacketActionHandlers, NexusPacketActionRegistryInput | support |

Extraction candidate queue

Recommended order, based on duplication, risk, and likely payoff:

1. Overlays and modal shells: initial shared modal shell and outcome/confirm primitives now exist under ui/overlays; remaining work should focus on auth/session gates, feature-status overlays, badge tooltips, and any modal content that deserves picker/outcome-specific composition. Keep auth/session logic separate from modal chrome.

2. Form field shells and searchable result lists: base segmented, inline-select, field shell, text input, text area, action row, and search-result primitives now live under ui/forms; Explorer search/export/import fields, identity lookup, locality create search dropdowns, trust notes, roles comments, and discussion composers now have shared presentation seams. Remaining work should target preference rows and broader route-local form sections.

3. Layout frames and section scaffolds: page/scroll frames, panel/workbench wrappers, panel headers, section bands, toolbar rows, and metric grids now live under ui/layout; identity page shell, dashboard/trust/roles metric rows, and a Explorer workbench panel have begun adopting them. Remaining work should target repeated route page wrappers, packet explorer panel shells, sidebar rail sections, and dashboard card rows.

4. Feedback states: scoped loading and basic empty/error/warning/status/operation cards now live under ui/feedback; remaining work should replace route-local loading, refreshing, empty, and outcome copy only where the shared card can preserve the existing layout.

5. Tab unification: keep Explorer document tabs as an extension of shared tab primitives. Do this after overlays/forms/layout so tab work does not become a broad behavioral rewrite.

6. Packet inspection/focus panels: focus, preview, dashboard, library, and Explorer all present packet summaries with related action/badge/provenance slots. Extract only after action/menu behavior stays stable.

7. Discussion feature promotion candidates: workspace panel shells, scrollable feed panels, thread toolbars, composers, vote/reaction pills, and recursive tree rails are now isolated under features/discussions; promote them into ui/* only after another surface needs the same skeleton.

8. Sidebar feature promotion candidates: rail toggles, compact nav rows, preference switch rows, grouped collapsible sections, scope/action row shells, and anchored compact action menus are now isolated under features/sidebar; promote them into ui/* only after another surface needs the same skeleton.

9. Identity feature promotion candidates: selectable identity rows/cards, passkey/session cards, and preference panels are now downstream candidates after the identity helpers moved into features/identity; promote only if another Nexus surface proves the same skeleton.

Suggested future folder map

‘‘txt app/components/nexus/ui/ actions/ # buttons, menu buttons, action menus, action lists, badge strips cards/ # base surfaces, packet cards, focus/preview cards, stat cards, warning/outcome cards tabs/ # shared tab frame/label/rail/stack plus Explorer document-tab extensions overlays/ # modal shell, confirmation, outcome dialog, popover, gates, tooltip hosts forms/ # field shell, text input, text area, composer, search box/result list, picker shell, toggles layout/ # page/scroll frame, section band/header, panel/workbench panel, panel split, rail section, toolbar frame, metric grid feedback/ # loading, empty, error, warning, operation result/status rows ‘‘

Guardrails for later migrations

- Preserve visuals and behavior first; extract structure before changing design.
- Do not move runtime/core logic into shared UI folders.
- Do not make packet-type-specific UI decisions inside generic templates. Generic templates receive slots, labels, state, handlers, and scope strings.
- Prefer one migration family per pass. A pass that touches overlays, tabs, forms, and route controllers at once is too wide.

- Update this hard inventory or the component catalog after each consolidation pass so the map does not rot into wall art.

Core Entities And Packet Model

Packet graph foundation

The packet graph remains the most robust unifying model.

Core packet rules:

- packet_id is the stable logical packet identity
- revision_id is the immutable exact revision identity
- parentrevisionrefs represent DAG ancestry
- schema_version tracks type-shape compatibility
- edges are the shared typed relationship collection
- authorityscoperef and applicablescoperefs stay separate
- revision_mode expresses type write semantics

Raw stored packets remain the historical signed fact. Adapted packets are runtime read views, and interpreted or read-model outputs are additional projections on top of those reads.

Core entities

Element

The universal identity anchor.

- person
- assembly
- organization
- service
- overlay or coordination group

Current direction:

- Element remains the durable actor, scope, and container type
- Element.subtype is the canonical classifier surface
- fresh Element packets reject the old top-level kind classifier
- dotted subtype forms such as person.claimed_identity or assembly.city are the forward model when the older shape previously depended on multiple classifier fields

Scope

The context or lens around an element. Every person can be treated as their own scope lens through You.

Assembly

A policy-governed civic or geographic element subtype, not a separate storage primitive.

Initiative

A generic Nexus concept for policy, template lineage, defaults, and work hierarchy. The pre-reseed reset treats Action(subtype: initiative) as the canonical top-level initiative anchor above campaign, program, mission, and task semantics. Cause is no longer an active fresh packet type.

Claim

The assertion and argument type.

Current forward direction:

- Claim can target packets generically
- Claim may carry claim_markdown and supporting refs
- Claim(subtype: relation_assertion) is the forward shape for claims specifically asserting a relation
- relation-specific claim semantics live under Claim(subtype: relationassertion).relationassertion.subtype

Current home-locality direction:

- canonical writes now use relation.residence.add
- that write produces Relation(subtype: residence) only; claims and reactions can be added around the relation, but are not automatically minted
- legacy residence.claim.set is retired from live fresh writes; historical Claim(residence) material remains readable through compatibility projections
- revise and withdraw semantics remain packet-native: status changes are represented by newly signed packet material rather than in-place mutation

Reaction

The evidence, certification, support, dispute, and packet-signal type.

Current code truth:

- claim support and dispute currently stay on Reaction
- Reaction and Claim are still distinct in code

- Reaction.subtype is the canonical reaction classifier

Role

A reusable role definition packet type. Exact-scope role assertions are currently represented through Claim(subtype: "relationassertion") with relationassertion.subtype: "role_association".

Policy

The configuration and legitimacy type for defaults, thresholds, and later execution rules.

Current direction:

- actual adopted or followed graph facts belong in Relation
- asserted or disputable statements belong in Claim
- evidence and support/dispute posture belong in Reaction
- legitimacy-sensitive relation rules belong in Policy.relation_requirements
- default inheritance belongs in Policy.default_policy, using packet refs for policies, templates, default packet sets, and preference material rather than runtime-only default names
- governance readiness belongs in Policy.governance_policy, reserving voter eligibility, trust stage, quorum, approval, vote method, and decision-report hooks without executing voting yet
- dependency requirements remain in Policy.dependencies_policy; subscriptions record what a subject accepts or excludes, and projections compare the two

Decision

A governance artifact type that exists in infrastructure and read surfaces, but whose real workflow semantics are still developing.

Converging civic grammar

The long-term Nexus direction is converging on a smaller reusable civic grammar:

- Element
- Relation
- Claim
- Reaction
- Report
- Action
- Policy

Current code already implements most of that grammar directly. Action now carries the forward initiative/work hierarchy and packet-backed default override refs, while Report has landed as a real packet type but its live use is still intentionally narrow.

Current code now ships the first concrete Report type use:

- Report(subtype: verification_report)
- Report(subtype: import_report)
- Report(subtype: decision_report) as reserved governance closure-report shape

This first live usage should still be understood narrowly. The type now exists as real packet infrastructure, but broader audit, incident, decision, or resolution report semantics remain later architectural work rather than implicit current product truth.

Packet-backed defaults

Defaults should remain packet-native rather than becoming Element fields or runtime constants. The current pre-reseed inheritance direction is:

- Definition(subtype: defaults_definition) parts carried alongside each packet definition
- bundle/default packet-set material
- initiative Action policy/template/default packet-set refs
- element policy/template refs or relations
- actor/client Preference

OWA-specific behavior should be represented by the default OWA Action(subtype: initiative) packet and its linked policies, templates, bundles, and preferences, not by special cases in generic packet schemas.

The default OWA seed now links its forward Action(subtype: initiative) anchor to default-inheritance and governance-baseline Policy packets. New default/policy resolution should use the Action anchor, then layer policy-selected defaultsdefinitionrefs and explicit overrides on top of definition-native defaults.

Schema evolution discipline

Before changing packet schemas, read this chapter first.

Also read docs/implementation-guide/trust-moderation-and-policy.md before changing Claim, Reaction, Relation, or Policy semantics.

For any packet type schema version change, the required checklist is:

- update the active schema or body shape
- update the type compatibility registry entry
- add or update upcast and downcast adapters where backward compatibility is intended

- update current schema version metadata
- update builders and type build definitions so new writes emit the canonical current shape
- update signature and write-preparation behavior if additive or defaulted fields affect compatibility or signing
- add or update tests for parse and read compatibility, adapted read behavior, write preparation, and any supported upcast or downcast path
- document the change in the relevant chapter and add a decision-log note when the change is architecture-significant

Relationships and graph semantics

The graph should continue to express relationships through typed refs and edges rather than UI-only linkage. Important relationship categories remain:

- references or depends_on
- proposes or supersedes
- belongs_to
- scoped_to
- endorsedby or vouchedby
- implements or enacts
- reports_on
- forkof or derivedfrom

Current scope-graph direction:

- canonical mounted ancestry prefers Relation(subtype: defaultancestryparent)
- canonical home-locality projection prefers Relation(subtype: residence); legitimacy evidence can attach through separate Claims and Reactions when policy or contestation requires it
- canonical follow relations now use Relation(subtype: follow) and are actor-only; legacy shell follow preferences are compatibility-only read input
- canonical association now uses Relation(subtype: association) without automatic claim wrapping, and associated scopes now count as mounted related scopes in shell projection
- canonical policy adoption now uses Relation(subtype: subscription) targeting a Policy packet rather than a separate adopts_policy relation subtype
- dependency requirements remain policy-layer semantics, usually Policy.dependenciespolicy, while dependson remains available only as a structural edge type rather than a Relation subtype

- subscription relations can carry subscription_options for inherited/default policies, dependencies, modules, templates, and default packet sets; excluding a required default does not erase the subscription, but projection should report partial alignment or review needs
- Relation(subtype: definedbylocation) is the live read seam for linked Location packets
- locality.path.create now emits locality Element packets, defaultancestryparent relations, provisional Location(subtype: region) packets, and definedbylocation relations together
- locality rows can now carry dynamic descriptor metadata, which is currently stored in linked Location.spatialpayload.scopedescriptor rather than through a packet schema bump
- legacy locality levels such as nation | region | city | district now function as compatibility buckets, while actual ancestry comes from the ordered path graph and not from a hardcoded four-slot ladder
- locality depth remains projection-only and should not be stored as a universal packet truth
- legacy parent_scope ancestry, shell follow preferences, and legacy Claim(residence) reads now belong in explicit compatibility projections or compatibility mirrors rather than inline main-path logic

Reaction

Reaction is the current packet type for target-agnostic responses. It replaces the prior split between standalone vote packets and reaction packets.

A reaction can carry independent channels for signal aggregation, support/dispute posture, and emotional response:

- vote_value: basic up/down signal used for visibility and formal vote-like aggregation
- attestation_value: support or dispute only
- emoji_keys: small emoji-key set for how people feel about a target

Relations, roles, proposals, discussion packets, and future action packets can all be reaction targets without the reaction needing packet-type-specific semantics.

Trust, Moderation, And Policy

Trust direction

Trust should remain layered, inspectable, and threshold-based rather than collapsing into a hidden score.

Current code truth:

- trust stages are currently explicit and threshold-based

- support and dispute evidence are packet-backed
- role posture is projected from scoped claims plus claim-targeted reactions

Direction:

- trust should stay scope-local
- trust should gate posting, voting, review, moderation, or role authority only through explicit policy and thresholds
- evidence and posture should remain inspectable even when defaults hide or deprioritize low-trust activity

Moderation and automoderation

Status: canon candidate

Current code truth

- moderation workflows are not yet fully implemented
- some read surfaces already expose packet-backed evidence and action visibility
- disabled placeholders are visible in several surfaces

Direction

- automoderation should be a generic feature, not a route-local trick
- default moderation baselines should come from the relevant element’s policy
- user-level moderation preferences should be available as adjustable policy or settings choices, similar in spirit to write protection preferences
- moderation should primarily act as visibility projection over packet and reaction graphs before it becomes hard rejection logic

Unresolved

- which moderation controls belong in generic policy versus OWA default policy
- which interactions should remain soft projection versus hard runtime enforcement
- how much moderation preference state should be actor-level policy versus client-level shell preference

Reactions, attestations, and claims

Status: canon candidate

Current code truth

- Relation is the structural type for adopted graph facts

- Claim is its own live packet type for assertions, arguments, and disputable relation claims
- Reaction is its own live packet type for evidence, certification, support/dispute, and packet-signal responses
- current discussion voting uses packet-signal reactions rather than a separate reaction type
- current claim support and dispute still run through the reaction service and reaction indexes

Direction

- keep Claim and Reaction distinct
- treat Claim as the forward layer for relation assertions and other packet-targeted arguments
- treat Reaction as the forward layer for supporting, disputing, certifying, or signaling around packets, including Claims and Relations
- do not require Claim wrappers for every Relation
- let policy require supporting Claims for legitimacy-sensitive Relations where needed

Current implementation note:

- the new Policy.relation_requirements seam exists so stricter relation-support rules can be expressed generically instead of being hardcoded as route-only logic
- Policy.defaultpolicy now carries packet-backed default refs for policies, templates, Definition(subtype: defaultsdefinition) packets, default packet sets, and preference material, plus explicit override paths; it must not introduce runtime-only default labels
- Policy.governance_policy now reserves packet-backed governance hooks for voter eligibility, minimum trust stage, quorum, approval threshold, vote method, and decision-report expectations
- this chapter should be read before changing Claim, Reaction, Relation, or Policy semantics because it owns the intended separation between assertion, evidence, graph structure, and policy requirements
- packet-native follow does not currently require a supporting claim in this phase
- packet-native association no longer auto-mints a supporting self-issued Claim(subtype: relation_assertion) alongside the structural relation

Current home-locality policy note:

- OWA-sensitive home-locality legitimacy resolves through Action(subtype: initiative) as the forward OWA policy/default anchor
- a canonical Relation(subtype: residence) is enough for default mounted home-locality projection; stricter Policy.relation_requirements can still require extra evidence for specific scopes

- the expected support model in this phase is separate Claims and Reactions attached around Relations only when evidence, dispute, or policy asks for them
- legacy claim-only home-locality reads remain compatibility projections, not the forward legitimacy model

Current dependency authority note:

- Definition dependencies_definition parts describe packet requirements and local engine contracts; runtime registries only validate and interpret those refs
- workflow dependency IDs must resolve to a Definition dependency part, Policy semantic, operation ontology entry, workflow resolver allowlist, or trusted local engine contract
- trusted runtime capability metadata is allowed only when it points back to packet meaning or an explicit local engine contract

Regulation and policy resolver direction

Status: initial trusted coordinator foundation exists; full governance language remains future work.

Policy packets exist today, and the Trusted Regulation Coordinator now provides the gated runtime surface for policy-oriented resolution. It resolves regulation contexts, policy contexts, write-policy gates, requirement lists, and readiness audits. Defaults and dependencies are now owned by the Trusted Planning Coordinator because they shape construction plans rather than enforce policy by themselves. Full proposal, voting, moderation, and role-gate semantics still need richer policy descriptors before governance becomes interactive.

The regulation resolver is designed to answer, progressively:

- who can create or mutate each packet type in a scope
- who can propose, vote, attest, review, or moderate
- which trust stage, role, or association gates apply
- which quorum, threshold, and vote method applies
- which policy-backed defaults are allowed or required when Planning asks
- whether proposal discussions, decisions, or decision reports should be created
- which packet types are allowed in a given scope

This resolver stays separate from builders and planners. Builders shape packets. Planning applies defaults, checks structural dependencies, selects builders, and composes operation plans. Regulation classifies the active policy envelope around an operation: allowed, required, blocking, advisory, definition-audit-only, or future-hook. Policy resolution is intentionally usable outside packet creation, including projection, import/export review, moderation, runtime reads, and governance checks.

Longer-term civic review framing

Direction:

- Claim should be understood as an unresolved assertion or request that invites evaluation, scrutiny, or action
- Reaction should be understood as a signed stance toward a target, such as support, dispute, certification, rejection, or abstention
- formal voting should route through governed Reaction aggregation inside a recognized process with eligibility and policy rules
- Report should be the long-term home for findings, outcomes, and context around a target
- a resolution or decision artifact should eventually read as a process-backed closure report rather than as unexplained authority
- quorum, minimum trust, eligibility, approval thresholds, and voting gates should be policy-backed defaults that can be inherited from the applicable initiative Action or scope policy and overridden by explicit packet refs

This framing is architectural direction, not a statement that all of those packet types or workflow surfaces already exist as live runtime behavior.

Verification truthfulness direction

Future verification UI and policy language should distinguish at least:

- structurally valid
- cryptographically verified
- provenance known
- locally trusted
- substantively true

Current product language does not yet fully expose those distinctions. The next verification/reporting work should avoid collapsing them all into one vague verified concept.

Current verification chapter truth:

- the runtime now signs verificationreport and importreport packets with a normal local validator identity
- those report packets are the signed record of what the node concluded
- a local runtime-owned verification index/cache exists so dashboard cards, Explorer search rows, export lookup rows, action menus, and Explorer verification views can project truthful current status quickly

- imported external verification reports are readable evidence, but local trusted status still comes only from the latest local validator report in this phase

Governance, Initiatives, And Decisions

Initiatives

Status: canon candidate

Current code truth

- initiative-specific filtering, subscriptions, and version visibility are not yet active product behavior
- current code does not yet implement initiative-version subscriptions or official versus unofficial filtering

Direction

- Action(subtype: initiative) is the forward top-level initiative packet for policy, template lineage, default packet sets, and later work hierarchy
- OWA should be modeled as an initiative Action inside Nexus, not as a hardcoded exception
- Cause is no longer an active fresh packet type
- lower work hierarchy levels should be represented through Action subtypes such as campaign, program, mission, and provisional task
- "official OWA" should mean conforming to recognized OWA dependencies, templates, and policies
- dependency requirements remain policy-layer semantics; depends_on is not a Relation subtype
- policy adoption is modeled by Relation(subtype: subscription) targeting a Policy, not by a separate adopts_policy relation subtype
- assemblies remain valid Nexus objects even when they fork or diverge from canonical OWA lineage

Unresolved

- the exact packet/API shape for initiative conformance and recognition beyond Action refs and policies
- how official, unofficial, derived, and forked initiative states should be encoded

Initiative versioning and subscriptions

Status: canon candidate

Direction

Initiative lineage should support version-aware adoption and viewing, including:

- current official version
- older official versions
- upgrade availability
- optional inclusion of unofficial forks

Subscriptions and default visibility should eventually support choices like:

- current official only
- all official versions
- official plus unofficial
- multiple initiatives together
- inherited default policies/dependencies with explicit deselection and visible partial-alignment states

Unresolved

- the exact version packet or metadata shape
- whether subscriptions belong primarily to actor policy, shell preference, or both
- how version compatibility and upgrade prompts should surface in Explorer and feeds

Assemblies, custody, and legitimacy

Status: canon candidate

Direction

- assemblies are elements with policy-governed civic semantics, not owned subsidiaries
- custody, legitimacy, policy authority, and governance outcome should remain distinct concepts
- association should remain a simple claim first; typed categories are unsupported
- multi-key assembly authority, custody transition, and protected-state mutation should remain explicit later design work

Unresolved

- the exact custody and multi-key packet model
- how authority grants or keyset policies should be represented
- how passed decisions eventually authorize protected state changes

Governance resolver direction

Status: canon candidate.

Governance should be built on packet-backed policy resolution, not hardcoded proposal or vote fields. Assembly-capable elements can receive default governance policies through Nexus/OWA default profiles. Those policies enable proposal behavior, define voting rules, and determine whether proposal discussions, decisions, or reports are generated by default.

The intended chain is:

- an assembly-capable element receives or adopts a governance policy
- the governance policy enables proposal creation and proposal discussion defaults
- proposal creation resolves the active governance policy before packet candidates are built
- votes or vote-like reactions target the proposal under that policy
- a decision packet records the formal outcome
- a decision report may record tally, evidence, and process context when policy requires it

OWA can provide governance presets and default policies. OWA should not override Nexus builders or redefine proposal, reaction, decision, or report packet anatomy.

Proposals and decisions

Status: canon candidate

Current code truth

- votes remain read-only
- proposals, votes, and decisions are not yet a fully interactive trusted workflow

Direction

- decisions should begin as legitimacy records and civic signals
- policy may later allow some decisions to activate real effects
- proposals should declare their intended outcome as explicitly and programmatically as possible
- quorum, eligibility, minimum trust, approval thresholds, and voting gates should come from packet-backed Policy defaults attached through the applicable initiative Action, scope, or proposal context rather than from hardcoded Vote fields
- a Decision should remain the formal outcome artifact, while Report(subtype: decision_report) is reserved for evidence, tally, and process-context closure material

Proposal outcome categories should eventually include:

- policy adoption

- mission or coordination plan
- resolution
- signal or poll
- other explicit action packages

Automatic execution should only be described as future behavior for explicitly supported built-in action handlers or canonical approval artifacts.

Unresolved

- which decision types can become effect-bearing first
- how multi-key assembly authority interacts with execution
- whether execution produces direct mutations, canonical approval packets, or both

Decision Log 2026-04

This monthly log condenses the April 2026 decisions that remain most important for current implementation and future migration work.

2026-04-08 foundations

- Packet identity uses stable packetid plus immutable revisionid.
- Revision history is a DAG with parentrevisionrefs, not a single chain.
- Packet relationships normalize into one typed edge collection.
- Element remains the identity-root type for people, assemblies, organizations, and services.
- The dedicated Nexus shell lives under /nexus/*.
- Function-first and scope-first remain one system rather than two route trees.

2026-04-09 shell and discussion shape

- Shared browser and Nexus query services over the packet graph became the projection seam.
- Active shell and scope routes moved onto packet-backed payloads instead of mock arrays.
- Discussions became a connected forum shell with feed, thread, and post workspaces.
- Feed cards became the primary thread-launch surface.
- Cursor paging and reply-branch controls became first-class route behavior.

2026-04-10 identity and auth

- Every graph-writing actor became a cryptographic Element(kind: "person"), including guests.

- Claimed auth uses local keys plus server challenge sessions rather than server-owned identity truth.
- Claimed auth now supports passkeys, rotating sessions, and explicit write-approval modes.
- Identity ceremonies moved into the Nexus shell.
- Reaction replaced packet votes as the reusable trust edge.

2026-04-17 architecture and trust pivot

- The final source roots became core, runtime, and app, with src/app as the Expo Router shell.
- Trust replaced Account as the top-level function surface.
- You became a first-class personal scope lens.
- Roles became a dedicated scope workspace.
- Role, Claim, and packet compatibility infrastructure became first-class runtime concerns.

2026-04-18 to 2026-04-20 locality and identity hardening

- Legacy signed identities remained valid after the roles-and-claims refactor.
- Home locality became the mounted geographic truth, while follow became shell preferences.
- Locality search and locality creation hardened around canonical geographic assembly flows.
- Location disclosure remained separate from mounted home-locality truth.

2026-04-23 and 2026-04-24 compatibility and Dispatch corridor

- Schema compatibility moved into a real raw/adapted/read-for-write pipeline.
- Dispatch pass 1 moved discussion write admission into core mutation law.
- Dispatch pass 2 introduced the shared prepare/sign/finalize corridor and packet-backed write-lock policy updates.

2026-04-25 to 2026-04-29 packet floor and verification

- Packet compatibility coverage is now classified explicitly.
- Legacy write seams were sealed before discussion-type cleanup.
- Raw packet signatures verify before adaptation.
- Live and near-live packet types now share one generic builder floor.

2026-04-30 surface-level runtime contracts

- Discussion workspaces now project action affordances through the shared NexusActionState and NexusActionIntentDescriptor contract.

- Packet Explorer became a shell-level read-only inspection workspace.

Decision Log 2026-05

2026-05-19 initiative action and discussion schema readiness

- Pre-reseed initiative semantics now point to Action(subtype: initiative) as the forward top-level initiative/work hierarchy above campaign, program, mission, and provisional task semantics.
- Action(subtype: initiative) is the fresh-reseed initiative/default anchor; Cause is pruned from active canon.
- Canonical discussions now reserve Discussion(subtype: post) for top-level forum artifacts that start threads, while reply chains use Discussion(subtype: message).
- Governance schema hooks remain policy-backed: quorum, minimum trust, voting gates, and decision-report closure semantics should be expressed through linked Policy/default material, Decision, and Report(subtype: decision_report).

This monthly log condenses the May 2026 decisions that remain most important for current public-surface and documentation shape.

2026-05-01 public surface convergence

- Shared public actions and section shells now route through one surface contract.
- PublicCardFrame separates shared graphic treatment from page-owned content layout.
- Support and Docs now share the same public frame and panel system.
- Public card frames gained ambient generated backgrounds owned by the shared frame layer.
- About surfaces joined the shared public frame path without absorbing card-specific animation rules.
- Public sections now use the universal card animation path.
- Public theme cleanup centralized lingering route-local styling values into named public seams.

2026-05 chaptering follow-up

- Canon docs are being split into index files plus shallow chapter folders to reduce edit cost, preserve top-level link stability, and keep current truth, durable architecture, and roadmap work separated cleanly.

2026-05 docs workflow cleanup

- The multi-chapter internal docs now treat chapter files as the canonical content source, while the top-level files remain short local index shells only.
- Public docs generation for implementation guide, specifications, and roadmap now compiles from chapter files only instead of concatenating the top-level shell files into the public artifact.
- The docs build now validates chaptered manifest entries, missing or duplicate source files, generated-artifact misuse, and shell-file drift before producing readable or downloadable outputs.

2026-05 shell honesty foundation

- Nexus now uses a shell-level early-access gate as a session-scoped entry warning rather than leaving public-facing instability as unstated product lore.
- The gate is intentionally lightweight and UI-owned: it is not packet-backed, not actor-specific, and not yet a tutorial flow.
- The same shell seam should later be able to host onboarding, release notes, or other blocking notices without redesigning the overlay stack.

2026-05 scoped Nexus loading foundation

- Nexus loading now has a UI-owned provider, boundary, overlay, and hook for caller-selected visual scopes.
- The loading layer intentionally ignores packet type, action registry identity, mutation intent, and runtime operation categories; UI boundaries decide what visual area is busy.
- A scope becomes active immediately to block duplicate input, while the visual overlay is delayed and minimum-duration guarded to avoid spinner flicker.
- Shared buttons and action-menu items can now opt into scoped loading with caller-supplied scope strings, while packet action registries and runtime operations remain loading-agnostic.

2026-05 Nexus UI folder foundation

- Nexus UI consolidation began with safe shared-module moves only: loading moved under ui/feedback/loading, action cards under ui/cards/action-card, action lists under ui/actions/action-list, and shared tab primitives under ui/tabs.
- The move establishes the canonical app/components/nexus/ui/* home without splitting nexus-ui.tsx, changing visuals, or moving feature-specific Explorer, locality, discussion, dashboard, preview, or focus components.

2026-05 Nexus overlay consolidation start

- Nexus overlay consolidation began with reusable modal chrome under app/components/nexus/ui/overlays/*, including a modal shell plus outcome, confirmation, and popover primitives.
- Dashboard packet validation, packet Explorer import outcomes, and locality create/reuse/picker dialogs now share the same modal shell while preserving their existing visual classes, close behavior, and route-local content.
- Auth/session gates and feature-status overlays remain specialized for later migration so generic overlay UI does not absorb session or shell behavior.

2026-05 Nexus search UI consolidation start

- Nexus search presentation now has shared primitives under app/components/nexus/ui/forms/search/* for search fields, result lists, result rows, status text, empty/error states, and loading-aware result boundaries.
- The consolidation is UI-only: Packet Explorer, identity, and locality create keep their own debounce timing, API calls, ranking, filtering, selection, and create-candidate behavior.
- Search loading scopes attach to the visual result surface, such as a dropdown or dedicated results panel, instead of packet types, route names, or search engine semantics.

2026-05 Nexus forms and feedback alignment start

- Nexus forms now have shared field shell, text input, text area, and field action-row primitives under app/components/nexus/ui/forms/*.
- Nexus feedback now has shared inline loading plus empty, error, warning, status, and operation-card primitives under app/components/nexus/ui/feedback/*.
- Identity fields, Packet Explorer import/export fields, trust association notes, and roles support comments began migrating to those primitives while preserving existing route behavior and keeping scoped loading caller-owned.

2026-05 Nexus layout and panel foundation start

- Nexus layout now has shared page/scroll frames, panel/workbench wrappers, panel headers, section bands, toolbar rows, and metric grids under app/components/nexus/ui/layout/*.
- Identity page chrome, dashboard/trust/roles metric rows, and a Packet Explorer search workbench panel began adopting these wrappers while preserving current visuals.
- Panel loading remains caller-owned by visual scope; layout components only provide a clean boundary mount point.

2026-05 Nexus UI primitive split

- The broad app/components/nexus/nexus-ui.tsx primitive file is now a compatibility bridge over focused app/components/nexus/ui/* component-type modules.
- Cards, actions, badges, forms, layout/chrome helpers, and attached tab rails now have canonical family homes while preserving their existing classes and behavior.
- Nexus callers have moved to the canonical @app/components/nexus/ui import path; future work should not add new primitive imports from nexus-ui.tsx.

2026-05 feature-status explainers

- Disabled and partial Nexus controls now route through a centralized feature-status registry instead of scattering one-off placeholder copy through route files.
- The first-pass status taxonomy is intentionally small: comingsoon, readonly, partial, and custom, with registry-owned overrides when a control needs more specific wording.
- Nexus shell now hosts one explainer card overlay at a time so disabled controls can stay compact while still opening truthful contextual explanations above Explorer, Library, Votes, and future surfaces.
- The shared NexusActionButton seam remains semantically disabled for these controls, but can now open an explainer instead of acting like a dead button when a feature-status id is attached.

2026-05 Explorer and shell mobile cleanup

- NexusInlineSelect now renders its open menu on a true overlay layer instead of trusting local stacking order, so Explorer View as menus can sit above adjacent header controls.
- Packet Explorer tab decks now behave like normal wrapped rows until they exceed three rows, and only then become scrollable.
- Narrow-screen shell entry now prioritizes immediate access to the real menu rails by opening both rails when the tray is opened, instead of preserving collapsed desktop rail state inside the mobile tray.
- That same mobile tray now sizes itself to the currently visible rails and closes entirely when both rails are minimized.
- Library packet highlighting is now treated as Explorer-linked state rather than a permanently sticky URL-only cue, so stale selection clears when the corresponding Explorer packet tab disappears.
- Explorer resize now lives in a dedicated Explorer-specific desktop drag controller rather than the main overlay coordinator, and uses global window mouse tracking plus temporary drag capture instead of a moving PanResponder handle.
- Shared browser packet projections now route through the core packet-title projection path instead of falling back directly to raw packet ids, which keeps Explorer and other packet-inspection surfaces from flipping loaded titles into percent-encoded route strings.

- Explorer reading now uses one primary content scroll surface with local collapsible header bands, while Close tabs has moved out of the shell command row and into the packet-tab deck as a packet-management utility.
- Explorer collapse controls were then tightened so the bands do not reserve their own chevron rows: collapsed sections disappear fully, and their restore actions move into the top shell command row as Packets and Views.

2026-05 Packet Explorer export workbench

- Packet Explorer Home is now explicitly split into Search and Export sub-tabs so future Explorer capabilities can grow from one stable workspace without collapsing back into the main packet inspector.
- The first live portability seam is export only: Search stays present but non-live, while Export becomes the active raw-json workbench.
- Packet export defaults to Raw packet, which means the current preferred revision envelope only; bundle wrapping is an explicit opt-in instead of the default artifact shape.
- Bundle export remains additive and transport-oriented: the envelope now records export metadata such as mode, root refs, counts, title, and note without changing packet schemas or mutating the packet envelopes inside packets.
- Phase 1 bundle scopes stay concrete and bounded to current graph/scoping contracts: packet history, outgoing references, incoming referrers, scope-stack ancestry, their combined union, and full local-store export as a separate node-level snapshot tool.
- Preview and download now share one export builder seam with explicit size guardrails so Explorer can show inline JSON only for small payloads while still allowing larger .json bundle downloads without inventing a second export format.

2026-05 Packet Explorer import workbench

- Packet Explorer Home now expands to Search, Import, and Export, with Search-card import shortcuts routing into the new Import workspace rather than spawning separate packet-vs-bundle pages.
- Import is intentionally a two-step flow: Analyze first, then Commit, so pasted or uploaded JSON does not mutate the local store until the runtime preview says the payload is structurally safe.
- Phase 2 import stays generic and backward-tolerant by accepting raw packet envelopes, exported bundle envelopes with packets, legacy bundle envelopes with revisions, and raw arrays, all normalized onto the existing bundle-import substrate.
- Web import now includes a browser .json file picker, but paste remains the universal fallback so the workflow still works outside the browser without adding a cross-platform file abstraction in this pass.

- Import analysis is structural rather than trust-verifying: it reports invalid entries, duplicates, missing parent revisions, type conflicts, affected packets, and likely open targets, then blocks commit on unsafe inputs instead of offering partial-import toggles.
- Post-import repair now preserves local preferred-head intent when divergence appears: if a newly imported branch creates multiple heads and the old preferred head still exists, Explorer restores that preferred revision instead of silently replacing it with the last imported head.

2026-05 Packet Explorer live search and export lookup

- Packet Explorer Search is now a real manual discovery workspace instead of a placeholder card, and its state stays Home-owned so query text, grouped results, and the active category survive normal Explorer tab movement while the session stays open.
- Search intentionally stays preferred/current only in this pass: it reads from the shared packet search index rather than adding FTS, graph traversal search, or historical revision fanout into the main result set.
- Exact revision-id search is still supported through a separate revision resolver seam on the packet store, which lets Explorer map historical revision IDs back to the owning packet while keeping the displayed packet card anchored to the current preferred revision surface.
- Search results are grouped into Direct, Name, and Text, with deterministic ranking and lightweight Open / Export routing instead of turning the first live pass into a broader action hub.
- Export now includes a compact active lookup when no packet target is preloaded, but that lookup intentionally remains narrower than the full Search workspace so Explorer keeps one discovery-oriented search surface and one operational packet-picker.

2026-05 Packet Explorer workspace cleanup

- Home workspace navigation has now moved out of the Home body and into the shared inspector row, so Search, Import, and Export behave like first-class Explorer modes rather than nested content tabs.
- The packet primary rail and the Home workspace rail now share one compact attached-tab treatment, dropping the previous secondary tab copy so the inspector band can stay horizontally accessible without running off-screen.
- Home-only dead controls were removed instead of being left as placeholder chrome, and the Search workspace now keeps only its core manual finder actions rather than duplicating Import navigation through extra shortcut buttons.
- Packet export was re-centered around direct packet lookup when no target is preloaded, with the packet-picker promoted ahead of explanatory copy so the export workspace feels operational instead of instructional.

- Search now defensively tolerates a missing runtime revision resolver by falling back to the preferred/current packet index path instead of failing the whole search request.

2026-05 Packet Explorer final polish

- Link traversal in Explorer is now explicit instead of ambiguous: grouped links expose Open in new tab, Open in current tab, and View in Library, while current-tab retargeting preserves the existing inspector state and refreshes the active packet identity fields.
- Packet tab decks now present Home as the workspace tab label, use middle truncation for packet titles, and surface desktop hover metadata so long packet titles and revision context stay discoverable without widening the deck.
- Home Export now includes an explicit Cancel reset path for packet-scoped export so preloaded packet targets can be cleared without leaving the Export workspace or disturbing the separate full local store export surface.
- Search preview caps and category browsing are now separated: All remains an 8-result grouped preview surface, while focused Direct, Name, and Text views page through larger result sets with server-backed 25-item pages.

2026-05 Dynamic scope graph consumer pass 1

- Home locality is now relation-first in the mutation corridor: canonical writes use relation.residence.add and produce Relation(subtype: residence) without automatically minting supporting claims.
- Legacy residence.claim.set, legacy Claim(residence), and legacy parent_scope ancestry remain readable only through named compatibility adapters and projections rather than through mixed main-path shell logic.
- Mounted scope projection now prefers packet-native structural relations such as defaultancestryparent and definedbylocation, while also exposing richer packet-backed shell metadata for later UI graph and dashboard work.
- OWA home-locality projection now resolves from packet-native relations sourced through the forward Action(subtype: initiative) anchor path; relation requirements remain available for stricter policies but are not the default fresh-write wrapper.

2026-05 Scope graph hardening and packet-native scope writers

- Scope ancestry resolution now explicitly surfaces structural graph state, including compatibilityparent, conflictingparents, cyclicancestry, and missingparent, instead of silently treating all non-canonical cases as equivalent.
- Follow is now canonical and actor-only: new writes use relation.follow.add / relation.follow.clear, while shell cookie follow remain a compatibility read bridge only.

- Association is now relation-first in the mutation corridor: canonical writes use relation.association.add / relation.association.clear, keep the supporting self-claim layer, and mount associated scopes as related mounted scopes rather than leaving them as badge-only side state.
- The first packet-native shell UI pass keeps the current rail layout but now groups scope context into Home, Associated, Followed, and Discoverable sections, roots the home trunk at You, and routes follow/associate sidebar actions through the canonical mutation corridor with a refetched shell projection afterward.
- locality.path.create now emits locality Element packets, canonical defaultancestryparent relations, provisional Location(subtype: region) packets, and definedbylocation relations in one signed packet batch, while parent_scope remains a named compatibility mirror only.

2026-05 shared packet-card actions and focus

- Dashboard preview lists and focused packet panels now share reusable packet action menus instead of route-local action clusters.
- Dashboard preview surfaces can focus one packet at a time locally, letting the focused presentation reuse the same packet action vocabulary without inventing a separate packet-detail route.
- Compact tooltip-capable icon badges are now part of the shared Nexus packet-card language instead of being dashboard-only decoration.

2026-05 sidebar follow-up polish

- The Home scope trunk now renders broadest-to-smallest, with You at the personal end rather than as the visual root.
- Sidebar rows no longer depend on inline badge clutter or left-side tree graphics for their primary meaning.
- Compact side overflow menus are now the active scope-row action pattern, with section placement carrying most of the relationship signal.

2026-05 Explorer truth clarification

- Packet Explorer Search, Import, and Export are live and should be treated as current product truth.
- Explorer remains read-only for packet mutation.
- Verification is now live for local packet validation, import reporting, and Explorer verification surfaces, while richer peer trust, provenance weighting, and broader report semantics remain later seams.

2026-05 verification chapter 1

- Report is now a real packet type in code, with verificationreport and importreport as the first live subtypes.
- The local runtime now bootstraps a normal signed validator identity and uses it to sign packet verification and import reports instead of introducing a privileged core-signature bypass.
- Packet Explorer now has a dedicated Verification rail, while dashboard and shared packet action menus can Validate, Revalidate, and View verification.
- Import now supports Validate first, Validate after, and Don’t validate, with validation reports and packet-level verification summaries recorded through runtime services and cached locally for truthful UI projection.

2026-05 verification hardening and bundle direction lock-in

- Packet-signature verification now accepts a narrow explicit set of identity-bearing signer packets, including Element(kind: person) and Element(kind: service), so the local validator stays a real service identity instead of a fake person-only compatibility hack.
- Verification summaries are now revision-anchored: local cache rows record the exact target revision and digest they verified, and current verified state only applies while that preferred revision still matches.
- Unknown signer status is now intentionally truthful: a packet can remain signed while still being unverifiable on the local node when the signer packet is unavailable, instead of being mislabeled as cryptographically valid.
- Explorer Import now exposes recent import_report history and richer artifact metadata through the existing report packet type rather than introducing an interim import-history packet type.
- The future bundle direction is now locked in at the roadmap level: Bundle will become a first-class packet type later, rebundling will create a new bundle packet rather than revising someone else’s bundle, bundle history should behave as a lineage graph, and legacy bundle JSON remains a runtime transport artifact rather than a core packet-compatibility type.

2026-05 verification tidy polish

- Explorer verification now surfaces freshness explicitly: the current preferred revision, the revision targeted by the latest local report, and whether that report is current, stale, or absent.
- Import History now speaks honestly about its current model: it shows the latest local import report per artifact identity rather than pretending to be a full append-only event ledger.

- The next major implementation chapter remains locality UX and geo standardization; a dedicated validation workflow screen and first-class Bundle packets stay intentionally unsupported for later work.

2026-05 locality UX pass 1

- /nexus/locality/create now has a true non-mutating Review path seam instead of pretending the write corridor itself is the review step.
- Top-level locality search now activates the existing create_candidate handoff so missing places can seed the broad-to-narrow builder directly, including from the Enter key path.
- Duplicate warnings in locality review are now actionable through Use existing, Edit name, and Create anyway rather than appearing only as passive blocker text.
- Use as home locality is now an explicit shared toggle for both selecting existing localities and creating new ones.
- The home-branch inclusion checklist is now visible during review as a UI-only preview affordance, while authoritative storage and projection of selected included scopes remains unsupported to a later locality pass.

2026-05 locality foundations phase 2A

- Locality planner semantics are now ordered-path first: the runtime interprets locality preview and create input as a broad-to-narrow path, and sparse paths no longer need to fill every legacy ladder slot.
- The current locality type picker is now real runtime input instead of decoration: path rows can carry LocalityScopeDescriptor data, with safe fallback descriptors derived from current legacy buckets when callers omit them during rollout.
- New locality writes now persist descriptor metadata inside linked Location.spatialpayload.scopedescriptor, while legacy nation | region | city | district values remain compatibility buckets rather than the forward locality ontology.
- Locality search and duplicate projection now preserve Unicode scripts, normalize accents safely for matching, and keep useful legacy ASCII-style aliases where that compatibility bridge is feasible.
- Home-tree inclusion persistence and projection fields remain intentionally unsupported to the next locality foundations pass rather than being bundled into descriptor and ancestry work.

2026-05 locality hardening pass 2A.5

- Locality preview and create now reject decreasing legacy compatibility bucket order such as city -> nation, while still treating ordered path ancestry rather than the ladder itself as the real graph truth.

- Descriptor reads now validate hierarchy_system against the allowed runtime enum so malformed imported Location metadata falls back safely instead of being trusted as-is.
- Top-level locality search can now still route into creation when same-name results exist in a different branch, preventing one existing Ontario from suppressing a new Ontario under another parent path.
- The create page now clears stale carried-over target rows more carefully when the target type changes across legacy buckets, and successful create/reuse completion resets the create builder before the success modal appears.
- Trust home-locality presentation has been tightened into a smaller onboarding-friendly card without changing the underlying home-locality actions or relation-first write path.

2026-05 locality runtime catch-up chapter

- Locality confirmation now routes through one composite locality.graph.apply mutation intent above locality.path.create, so structural locality writes, selected home-locality changes, scope associations, follow, and display preferences are coordinated together instead of being chained from the client.
- Partial success is now explicit at that runtime seam: structural locality planning and packet writes remain phase one, while relation and display-preference writes report their own outcomes without pretending the whole flow is all-or-nothing.
- Actor-to-scope relationship reads are now centralized through one runtime controller that treats canonical residence, association, and follow relations as the main truth, while preserving guest and compatibility follow behavior as an explicit fallback bridge.
- main is a visible-scope preference rather than a relation, and associated/followed parent-context display toggles are persisted alongside it through the current claimed-actor preference bridge.
- The generic scope-graph projection now returns server-projected home, associated, followed, main, and discoverable sections for the sidebar, and OWA-specific initiative-anchor relation-policy lookup has been moved out of the generic graph core into a narrower adapter layer.

2026-05 Preference.element packet write corridor

- Preference is now enrolled in the canonical packet ontology as a replaceable packet type, with Preference.element carrying actor-owned scope-display preferences.
- Claimed actor scope-display writes now create live Preference.element packet revisions and also update the legacy runtime preference table as a compatibility cache.
- Scope-display reads prefer the latest active Preference.element packet and fall back to the legacy table when no packet exists; guest preferences remain cookie/session compatibility state.

- Claimed shell preference reads now resolve the actor from the authenticated session; actor query mismatches use guest projection instead of reading another actor’s private preference packet.
- Preference.element now includes an interface.shell_chrome section for navigation mode, theme mode, and UI density so remaining shell-interface preferences can move into the same body without redefining the packet shape.
- runtime/nexus/server/packet-runtime-master-handler.ts is the first generic runtime connector corridor: routes can request a connector id, the handler resolves the packet definition and mutation action plan, then dispatches to local trusted connector code instead of hardcoding route-specific packet writes.
- preference.element.interface.set was initially enrolled in that master handler as a live bridge. It wrote Preference.element.value.interface.scopedisplay, could preserve or update interface.shellchrome, and kept the legacy scope-display table as a compatibility cache. The later Dispatch promotion pass moves claimed writes to preference.element.set and keeps this connector runtime-ready.
- The manifest definition Dispatch metadata bridge remains non-executing and descriptor-only. The new runtime master handler still runs trusted local connector code; it does not execute arbitrary behavior from imported definitions.

2026-05 policy/dependency semantic authority

- Policy current schema now includes nullable defaultpolicy and governancepolicy sections so default inheritance and governance readiness are packet-backed before reseed design.
- Dependency refs in Definition parts and workflow plans now resolve through packet dependency semantics, Policy semantics, operation ontology entries, trusted workflow resolvers, or explicit local engine contracts instead of loose runtime-only strings.
- The seeded OWA Action(subtype: initiative) is now the policy/default anchor and links to default-inheritance and governance-baseline policies.

2026-05-20: Canonical subtype reset before reseed

- Active packet canon is pruned to Definition, Bundle, Element, Location, Role, Claim, Relation, Report, Proposal, Reaction, Decision, Action, Discussion, Policy, and Preference.
- Cause, split discussion types, split initiative/work types, Signal, Minutes, Artifact, and other unused alpha types are removed from fresh packet canon.
- Fresh active packet bodies use top-level body.subtype as the packet classifier. Old top-level classifier names are treated as alpha archive shapes, not fresh write compatibility obligations.

- Reseed continuity will preserve identity/key continuity by default; stale packet-id relations, home locality, follow, associations, main-tree scope IDs, and old discussion placement are not carried forward by default.

2026-05 final pre-reseed wrap-up

- Live fresh writes no longer accept association.claim.set or residence.claim.set; canonical association and home-locality writes now use relation-first mutation intents only.
- Legacy claim/home-locality material remains compatibility-readable and importable, but it is no longer a live mutation corridor entrypoint.
- A final pre-reseed readiness report now inventories canonical write intents, compatibility-only surfaces, OWA default anchors, required seed policies, discussion defaults, canonical definition packet types, and out-of-scope packet types for the separate reseed design pass.

2026-05 definition packetization and Preference Dispatch promotion

- Active manifest definition parts now produce schema-validated canonical Definition packet envelopes, and those envelopes are grouped into one canonical Bundle.packet_set definition profile inventory for reseed readiness.
- The seeded definition profile audit compares packet material back to the core manifest and fails closed on missing parts, unexpected parts, stale profile metadata, duplicate bundle refs, or digest drift.
- Stored Definition and Bundle packet material is now reseed truth material, but execution remains trusted-local; imported definition packets may describe schemas, operations, policies, dependencies, planners, and builders, but they cannot introduce server code.
- Definition, Bundle, and Preference are now canonical packet types with body schemas, compatibility entries, builder support, pipeline inventory coverage, and packet-store validation like other active packet types.
- Claimed shell preference writes now run through the standard signed mutation prepare/finalize corridor as preference.element.set, preserving projected responses, actor proof/session checks, same-value no-op behavior, and legacy cache sync. /api/nexus/shell-preferences remains guest compatibility state only.
- The direct preference.element.interface.set runtime connector is retained as a definition/internal comparison bridge rather than the live claimed-write corridor.

2026-05 definition kernel and stored profile decision

- Nexus will hardcode only the Definition packet kernel needed to cold-start, validate, and compose definition material. This kernel includes the Definition body schema, supported Definition part subtypes, and the bootstrap resolver for turning Definition parts into a local active definition view.

- Long-term packet type authority should live as stored Definition packets, not TypeScript-only descriptors. TypeScript packet definitions remain bootstrap/compiler seed material, local fallback material, and test fixtures until active Definition profiles can be resolved from packet storage.
- A packet type definition is a profile assembled from Definition part packets, not one monolithic object. Schema, actions, builders, planners, projections, compatibility, defaults, and dependencies may version or fork as separate Definition parts and travel together in a Bundle.packet_set profile inventory.
- Nodes/scopes will eventually select their active definition profile through packet-backed defaults, preferences, and policies. The current Trusted Definition Coordinator already models source kinds, trust tiers, pins/preferences, ranking, compatibility-only candidates, quarantine, and ignored candidates, but archive-backed profile loading remains a future migration seam.
- Imported Definition packets remain descriptive only. They can declare schemas, actions, builders, planners, projections, defaults, policies/dependencies, and compatibility metadata, but execution must stay inside trusted local runtime capabilities and allowlisted coordinators.

2026-05 Nexus discussions UI decomposition

- Nexus discussions began moving route-local UI into app/components/nexus/features/discussions/* while preserving src/app/nexus/discussions.tsx as the route/controller owner for query state, loading, mutations, auth gates, and reply branch state.
- Extracted discussion feature components now cover feed post cards, root post cards, vote/reply-count pills, recursive reply trees, and post/reply composers.
- Discussion loading adoption remains visual-scope owned: feed load-more, root replies, reply branches, vote pills, and composers can mount scoped loading boundaries without packet-action registry or runtime coupling.
- Candidate universal pieces such as composer shells, vote/reaction skeletons, and tree rails remain feature-local until another Nexus surface proves the same reusable shape.

2026-05 Nexus discussions workspace panel extraction

- Discussion feed, thread, and post workspace sections now have feature-local panels under app/components/nexus/features/discussions/*, including a thread toolbar, while the route keeps router, auth, loading, mutation, and reply-state ownership.
- The panel props are explicit callbacks and display state rather than runtime services or router hooks, keeping the feature components ready for later generic promotion if another surface needs the same workspace/feed/thread/composer skeletons.
- Scoped loading boundaries remain attached to visual regions such as feed, root replies, reply branches, votes, and composers; operation semantics still stay outside UI.

2026-05 Nexus sidebar feature extraction

- Nexus sidebar rail/menu UI began moving into app/components/nexus/features/sidebar/* while nexus-sidebar.tsx remains the shell controller for router, auth, preference persistence, rail animation, and scope mutation state.
- Extracted sidebar feature components now cover rail toggles, guest avatar, preference switches, current context card, function menu content, scope section headers, grouped scope rows, scope action menus, and scope menu content.
- Scope follow, association, and main-tree visibility actions now use caller-owned loading scopes such as sidebar:scope-follow:<scopeId> and block the affected scope row/menu region without coupling loading to packet/runtime semantics.
- Candidate universal pieces such as rail toggles, compact nav rows, preference switches, grouped sections, scope/action rows, and anchored compact menus remain feature-local until another Nexus surface proves the same reusable shape.

2026-05 Nexus locality create UI decomposition

- Nexus locality create UI began moving into app/components/nexus/features/locality/* while src/app/nexus/locality/create.tsx remains the controller for params, auth gates, draft persistence, graph/path state, API calls, mutation handlers, and modal state.
- Extracted locality feature components now cover the search panel, builder panel, parent/type/create-kind dialogs, selected-result and remove confirmation dialogs, outcome dialogs, graph rows, and preview panel.
- Locality loading remains visual-scope owned: search results, parent picker results, graph row results, preview, create-path, and set-home actions use caller-owned loading scopes without coupling UI to packet/runtime semantics.
- The obsolete level-row ladder builder path was removed after extraction; the active create flow is the graph-based builder, with route/controller ownership preserved.

2026-05 Nexus Explorer feature extraction

- Packet Explorer moved into app/components/nexus/features/explorer/*, including its shell overlay, document tabs, toolbar, search, import/export, data, links, resize, and inspection panels.
- Explorer import/export work now uses caller-owned visual loading scopes for packet export, store export, import analysis, import commit, and import history, while preserving existing disabled labels and panel behavior.
- Explorer document tabs remain feature-local and compose shared tab primitives only where they already did; generic document-tab promotion is deferred until a dedicated tab-extension pass.

2026-05 Nexus Explorer internal decomposition

- Explorer import cards, export preview/request helpers, packet inspection panels, and the validation dialog were split into focused feature-local files under app/components/nexus/features/explorer/*.
- Packet validation now has a caller-owned visual loading scope, packet-explorer:validation:<packetId>, mounted around the validation panel without coupling loading to packet/runtime semantics.
- The main Explorer shell still owns tabs, search state, packet loading, validation handlers, resize behavior, router calls, and session coordination; generic promotion remains deferred until repeated component types appear elsewhere.

2026-05 Nexus primitive adoption checkpoint

- Nexus entered a primitive-adoption checkpoint phase after the main ui/* foundations and feature extractions, classifying remaining raw React Native usage as official primitive internals, acceptable feature-local controls, route/controller usage to migrate, or behavior-sensitive deferred usage.
- The first low-risk cleanup converted only trivial outer route scroll shells in account, dashboard, discussions, roles, trust, and votes to NexusScrollFrame; scroll containers with refs, handlers, bounded modal regions, graph focus, Explorer resize/session behavior, or sidebar shell animation remain local.
- Stale @app/components/nexus/nexus-ui imports are treated as retired outside the compatibility bridge. Loading adoption remains visual-scope owned, with future route adoption limited to places where the async owner and blocked visual boundary are obvious.

2026-05 Nexus UI pre-audit wrap-up

- Identity route UI helpers moved into app/components/nexus/features/identity/*, completing the main feature-folder map alongside discussions, Explorer, locality, and sidebar.
- app/components/nexus/nexus-identity-ui.tsx remains a temporary bridge only; identity routes now import from the feature folder while keeping router, auth/session, passkey, storage, mutation, and error behavior route-owned.
- The wrap-up stayed organizational and audit-oriented: deeper trust/roles extraction, sidebar second split, dashboard/library/account/votes polish, and interface event coordination work remain post-audit follow-ups.

2026-05 Interface Event Coordinator and Trusted Dispatch foundation

- The client-side Interface Signal Conductor direction was renamed to Interface Event Coordinator, reflecting its role as the master interface event lifecycle controller rather than a trusted runtime actor.
- Trusted Dispatch Coordinator is now the canonical runtime front desk name for request normalization and client-intent preflight; the existing Trusted Request Coordinator remains as the compatibility implementation bridge.
- Mutation prepare now passes through dispatch normalization and fail-closed client-intent preflight before the existing Dispatch corridor continues. Mutation finalize records dispatch normalization while preserving ticket-based finalize preflight and signed payload schemas.

2026-05 Trusted runtime coordinator audit and caller migration

- The trusted coordinator scaffold audit now checks canonical result kinds, top-level barrel exports, and public method drift; Resolution remains the only accepted legacy-flat coordinator warning.
- Definition, Regulation, and Planning result envelopes now use their canonical coordinator kinds, with older aliases retained only for legacy workflow paths.
- Packet Explorer bundle export and import commit now route through Trusted Exchange and Trusted Archive while preserving existing Explorer payloads, import reports, validation modes, and preferred-head repair.
- The verification service is now a compatibility wrapper over Trusted Verification for packet assessment, while local validator identity and report-writing stay service-owned until a later Certification/Archive migration.

2026-05 Trusted process chains and issue taxonomy

- Trusted coordinator results now support optional lightweight process chains that record stage-by-stage runtime diagnostics without automatically creating report packets.
- The trusted issue taxonomy uses canonical dotted codes while mapping older underscore codes as aliases, giving reports and future interface handling stable error categories.
- Archive and Exchange are the first deeper adopters: Archive records partial write progress, failed/skipped work, and preserved partial results; Exchange chains preview/plan/commit/export stages and child Archive results.
- Report packet persistence is intentionally deferred. V1 only provides a process report draft helper so later Certification/Archive/report flows can choose when to sign and store diagnostics.

2026-05 Runtime cleanup bounded-context foundation

- Runtime cleanup began after the major trusted coordinator seams stabilized, with runtime/nexus/server/* moving toward bounded-context folders instead of one flat service directory.
- Identity, discussion, reaction, locality, scope, Packet Explorer, readiness, and shared helper modules now have canonical homes with top-level compatibility bridges preserved for existing callers.
- Dispatch mutation flow owns the reserved bounded-context home; signed corridor helpers should stay behavior-preserving and ticket/signing paths easy to verify.
- The rule for this chapter is move-first and split-later: large services keep behavior intact now, then decompose by responsibility inside their bounded-context folders in later passes.

2026-05 Runtime responsibility and ideal ownership audit

- Runtime cleanup shifted from folder organization to ideal ownership analysis, classifying runtime processes by whether they belong to Trusted Coordinators, packet-definition metadata, Projection, storage adapters, OWA adapters, compatibility bridges, or the legacy signed corridor.
- Discussion, reaction, locality, and scope runtime services are treated as transitional product/runtime adapters rather than final generic architecture.
- Signed-corridor naming is now explicitly legacy signed-corridor language; new work should use Dispatch, Regulation, Planning, Certification, Archive, and Interface Event Coordinator ownership terms where possible.
- The audit records packet-definition opportunities for projection fields, action availability, policy requirements, build/default/dependency assumptions, and import/export/verification summaries without allowing imported packet definitions to execute local runtime behavior.

2026-05 Dispatch-owned write pipeline correction

- The mistaken Trusted Write coordinator direction was rejected: write lifecycle orchestration belongs to Dispatch, not a new peer coordinator.
- /api/nexus/mutations/prepare and /api/nexus/mutations/finalize now call Dispatch-owned write lifecycle methods and do not call NexusMutationService as route-facing authority.
- relation.follow.add is the first live write to complete the corrected Dispatch-owned chain: Planning, Building, Inspection, Certification ticketing, signed-packet bundle certification, Verification, and Archive storage.
- relation.association.add now uses the same full Dispatch-owned chain, with generic scoped Relation materialization and finalize-time mutation-kind parity checks.

- Other mutation intents remain coordinator capability gaps until they receive equivalent packet-envelope materialization, Certification signed-bundle checks, Verification handoff, Archive storage, and result projection. They must not fall back to the legacy signed corridor.

2026-05 Trusted runtime audit guardrail housekeeping

- Added package-level trusted coordinator test scripts: test:trusted-coordinators for the runtime coordinator tests and check:trusted-coordinators for audit plus tests.
- Hardened the trusted coordinator audit to catch unmanifested trusted*coordinator folders, missing trusted test scripts, manifest/barrel/method/result-kind drift, and unregistered trusted issue codes.
- Runtime crossing notes now recursively scan real implementation folders under runtime/nexus/server/ and src/app/api/nexus/ instead of only top-level compatibility bridge files. Direct storage touches, signature verification, packet interpretation, bundle import/export, API packet parsing, and legacy Dispatch corridor references remain non-failing migration notes.
- Removed the empty trustedwritecoordinator remnant. Writes remain Dispatch-owned lifecycle orchestration, not a separate coordinator.
- Corrected the Exchange manifest note: Exchange can orchestrate accepted import commits through Archive, but Archive remains the storage owner and Exchange should not bypass Compatibility or Verification ownership.

2026-05 Trusted Exchange import commit hardening

- Exchange import preview now normalizes JSON string inputs and archive byte payloads into a consistent packet bundle before asking Verification and Compatibility for posture.
- Exchange import commit now narrows Archive writes to accepted plan entries only; skipped duplicates and blocked/manual entries are no longer passed through to Archive as part of the commit bundle.
- Verification and compatibility risk acknowledgements are explicit commit inputs. Packet Explorer currently treats an approved commit from its preview flow as acknowledgement so existing UI behavior is preserved.
- Exchange commit receipts now expose planned, archived, skipped, imported, missing, and unexpected revision-key counts/lists so Archive import results can be compared against the Exchange plan.
- The trusted issue taxonomy gained Exchange archive-import mismatch/failure aliases so coordinator audits fail if future code invents unregistered runtime issue codes.

2026-05 Dispatch finalize certification/verification hardening

- /api/nexus/mutations/finalize now passes submitted signed packet material through to Dispatch without route-level packet-envelope parsing, preserving the raw signed material for Trusted Verification.
- Certification accepts the signed bundle shape, parses internally for ticket/digest/type/signer matching, rejects duplicate packet revision keys, and records the exact certified packet revision keys on the certified packet set.
- Dispatch now verifies the raw signed packets through Trusted Verification, compares verified packet revision keys against Certification before Archive, derives the finalized mutation kind from the certified plan, and rejects mismatched caller-supplied intent labels.
- Archive now requires certified packet revision keys and checks that extracted archive-ready envelopes match those keys before any write, so Archive stores only the certified packet set.
- The trusted issue taxonomy gained canonical entries for certified-key and finalize mismatch blockers so audit remains fail-closed for invented issue codes.

2026-05 Dispatch reaction vote write migration

- reaction.vote.set now enters the Dispatch-owned write lifecycle with Reaction packet planning hints, body materialization metadata, generic Reaction packet envelope building, Inspection, Certification, Verification, and Archive storage.
- Dispatch derives reaction.vote.set from both workflow-backed and trusted operation plan IDs so finalize-time mutation-kind parity checks cover reaction writes like relation writes.
- Finalize response decoration refreshes the existing reaction derived index/summary after Archive for parity with the current discussion UI, but that bridge now lives in the reaction runtime adapter instead of Trusted Dispatch. This is still not the final Projection-owned read-model architecture.
- Inspection now ignores reserved _trusted* materialization metadata keys when comparing planned body values to the candidate body, allowing Dispatch/Building to carry header materialization inputs without polluting packet bodies.
- The runtime cleanup tab now records reaction vote migration as completed for the legacy signed-corridor checklist while keeping remaining mutation intents open.

2026-05 Legacy mutation service containment and reaction finalize adapter

- NexusMutationService is no longer constructed by createNexusPacketServiceRegistry or exposed on NexusPacketServices; the remaining mutation/signed-corridor files are legacy corridor candidates rather than live service-graph dependencies.

- Trusted Dispatch now returns generic trusted finalize facts for reaction.vote.set instead of importing SQLiteReactionService or refreshing reaction-derived state directly.
- The finalize API route decorates completed reaction.vote.set responses through reaction-finalize-response-adapter, preserving the current target/value/summary response compatibility while keeping reaction runtime services outside the trusted coordinator.
- The trusted coordinator audit now fails if the live service registry reintroduces NexusMutationService, if prepare/finalize routes call legacy mutation service methods, or if Trusted Dispatch imports reaction runtime services.

2026-05 Packet Explorer projection migration pass

- Packet Explorer’s generic inspector payload now uses Trusted Archive for preferred/head revision resolution and graph edge lookup instead of direct packet-store/query-service calls.
- The inspector read-model panel now uses Trusted Projection output as readmodelview, preserving the existing response field while removing the direct legacy packet-interpreter call from Packet Explorer data.
- Scope labels and related link labels in the inspector are resolved through archived packet projections. Search, Export, Import, discussion, reaction, scope, and locality read models remain separate migration lanes.

2026-05 Projection card-list bridge migration

- Trusted Projection now exposes resolvePacketCardListProjection for already-selected packet cards, keeping query selection, ranking, and scope filtering outside Projection while centralizing card/list presentation shaping.
- Packet Explorer Search preserves its direct/name/text grouping, scoring, pagination, and response fields, but selected result rows now pass through the Projection card-list seam before being mapped back to the existing payload shape.
- nexus-query-data now projects generic dashboard, votes, and library packet-card lists after nexusQueryService has already selected the cards. Discussion, reaction, scope, locality, and deeper product-specific route read models remain separate adapter cleanup lanes.

2026-05 Legacy mutation executor deletion

- Removed the old route executor stack: NexusMutationService, Dispatch prepare/finalize handlers, old handler domain maps, mutation prepare/finalize handler maps, signed-packet finalizer, preference Dispatch workflow, and manifest Dispatch metadata bridge files.
- Trusted Dispatch remains the route-facing prepare/finalize authority. Deleted executor files are now guarded by audit:trusted-coordinators so they fail if restored.

- Static genericization/handoff/readiness ledgers remain temporarily because they describe migration state and pre-reseed closure, but their text now points to Dispatch/trusted-chain authority rather than the old Dispatch corridor.
- Direct-write seam classification now treats discussion/reaction/preference helpers as transitional adapter/internal or compatibility-cache writes instead of Dispatch-internal helpers.

2026-05-30 - Packet-backed definition profile preference carrier bridge

- Trusted Definition remains the owner of active definition source selection. No new runtime service was added.
- ResolveTrustedDefinitionContextInput can now accept packet-backed definition profile preference carriers through definitionprofilepreferencepackets. The current bridge reads active Bundle.packetset bodies whose bundledata.definitionprofile_preferences descriptors normalize into TrustedDefinitionRuntimePreference records before candidate ranking.
- Preference targeting now carries both nodeelementid and scopepacketid, so a carrier can select a profile for one node, one scope, or broad local runtime use without hardcoding packet-specific behavior in core.
- This is still a bridge, not the final archive search path: local archive / pinned bundle discovery should feed this same coordinator input after reseed material is stored. Imported definition material remains descriptive; trusted local coordinators still own execution.

2026-05-30 - Archive-backed definition profile preference discovery bridge

- Trusted Definition now has an archive-backed profile preference discovery path inside the existing coordinator surface: resolveContextWithArchiveProfilePreferences queries Bundle packets through Trusted Archive, reads candidate carriers through Trusted Archive, and then feeds the discovered carriers into the existing packet-backed preference normalizer.
- This keeps ownership clean: Archive owns packet-store reads, Definition owns source/profile selection, and the shared definition preference normalizer remains the single conversion seam into TrustedDefinitionRuntimePreference records.
- The normal synchronous Definition resolver remains intact for bootstrap, tests, and callers that already have carriers in hand. Archive-backed discovery is async because it crosses the Archive read boundary.
- This is still a narrow bridge for definition profile preference carriers. Full local-archive/pinned-bundle Definition candidate loading should reuse the same coordinator path later instead of adding new runtime services.